SAPIENZA
UNIVERSITÀ DI ROMA

Faculty of Information Engineering, Informatics and Statistics
Department of Computer Science

Concurrent Systems

Author:
Simone Lidonnici

7 may 2026

Contents

1

Mutual Exclusion

1.1 Concurrency

A Concurrent System is a set of sequential state machines that run simultaneously and interact through a shared medium. The goal is to combine the work of different processes, that solve different tasks in parallel.
We will focus on concurrent systems that have 3 main properties:

- **Reliable**: every process correctly executes its program.
- **Asynchronous**: each process has its own clock.
- **Shared memory**: every process has a local memory but there are some registers that can be accesses by every process.

The number of processors that we have is not relevant so we will assume we have one for every process.
Synchronization of a concurrent systems means that the behaviour of one process depends on the behaviour of the other processes. There are two principal interactions:

- Cooperation: different processes work to make all succeed in their task.
- Competition: different processes aims to execute the same action, but only some of them succeed. This usually happens when trying to access a shared resource.

1.2 Mutex definition

A mutex ensures that a part of the code is executed as if it was atomic, so without other processes running in between. The mutex is needed only in code parts that interact with shared data.

Critical section

A **critical section (CS)** is a set of code parts that must be excuted without interferences. When a process is in a CS, no other process can be in a CS for the same shared object.

The mutex problem is designing an entry protocol **lock** and an exit protocol **unlock**, such that the code encapsulated between this two protocols can be run by at most one process at a time.

Every mutex should satisfy 2 properties:

- **Safety**: there is at most one process in a CS.
- **Liveness**: something good eventually happens. This property has various options:
  - **Deadlock freedom**: if there is at least one invocation of lock, eventually after at least one process enters a CS.
  - **Starvation freedom**: every invocation of lock eventually grants access to the associated CS.
  - **Bounded bypass**: with n processes, there exists $f : N \to N$ such that every lock enters the CS after at most $f(n)$ other CS (processes).

The liveness properties have a hierarchy such that Bounded bypass implies Starvation freedom that also implies Deadlock freedom.

1.3 Atomic Read/Write registers

Atomic Read/Write registers are storage units that can be accessed through two operations (`READ` and `WRITE`) such that:

1. Each invocation of an operation:
   - looks instantaneous and can be considered as a single point on the timeline (there exists a function $t : OpInv \to \mathbb{R}^+$)
   - may be located in any point between its starting and ending time (we have that $t(opInv) \in [t_{start}(opInv), t_{end}(opInv)]$)
   - does not happen together with any other operation ($t(opInv) \neq t(opInv')$)
2. Every `READ` returns the preceeding value written in the register.

There are different types of registers based on whether they can be read and written by one or many different processes:

- single-read/single-write (SRSW)
- single-read/multiple-write (SRMW)
- multiple-read/single-write (MRSW)
- multiple-read/multiple-write (MRMW)

1.3.1 Peterson algorithm

The Peterson algorithm designs a mutex for 2 processes:

Algorithm: Peterson algorithm (2 processes)

```
// Inizialize FLAG[i] as false
def lock(i):
    FLAG[i] = true
    LAST = i
    wait (FLAG[i-1] = false or LAST != i)
    return

def unlock(i):
    FLAG[i] = false
    return
```

This algorithm requires 2 one-bit MRSW registers for the `FLAG` and 1 one-bit MRMW register for `LAST`. Every lock-unlock requires 5 accesses to the registers.

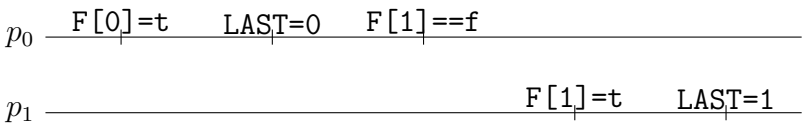
Peterson algorithm properties (2 processes)

The Peterson algorithm designs a working mutex and has the bounded bypass property with bound 1.

Proof of safety:

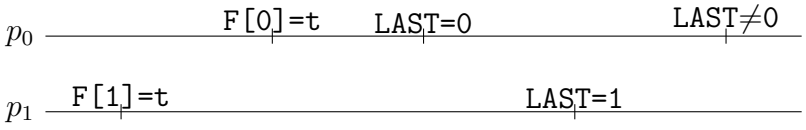
We can prove that the algorithm works by contraddiction, assuming that the two processes p_0 and p_1 are inside the CS. How has p_0 entered the CS:

1. If `FLAG[i]=false` that is possible with this sequence:



   With this sequence p_1 cannot be in the CS because `LAST=1` and `FLAG[0]=true`.

2. If `LAST=1` that is possible with this sequence:



   With this sequence p_1 cannot be in the CS because `LAST=1` and `FLAG[0]=true`.

Proof of liveness:
Let p_0 invock lock, if it wins waits 0, otherwise must be `FLAG[1]=true` and `LAST=0`.
`FLAG[1]=true` means that p_1 has invocked the lock and will eventually enter the CS and unlock, putting the flag down. Now p_1 could invock the lock again:

- If it never locks again p_0 will eventually read the flag down and enter the CS, waiting 1.
- If p_1 locks again:
  - If p_0 reads `FLAG[1]` before p_1 locks again, then p_0 enters the CS, waiting 1.
  - If p_0 doesn't read before p_1 locks again, then p_1 will set `LAST=1` and will wait, so p_0 will eventually read `LAST` and enter the CS, waiting 1.

There is also a Peterson algorithm for an arbitrary number of processes n, using levels:

Algorithm: Peterson algorithm (n processes)

```
// Inizialize FLAG[i] as false
def lock(i):
    for lev in [1,n-1] :
        FLAG[i] = lev
        LAST[lev] = i
        wait (∀k FLAG[k] < lev or LAST[lev] != i)
    return

def unlock(i):
    FLAG[i] = 0
    return
```

This algorithm require n MRSW registers with $\lceil \log n \rceil$ bits for `FLAG` and $n-1$ MRSW registers with $\lceil \log n \rceil$ bits for `LAST`. Every lock-unlock requires $(n-1)(n+2)$ accesses to the registers, so has a cost $O(n^2)$.

Peterson algorithm properties (n processes)

For every level $l \in [0, n-1]$, there are at most $n-l$ processes in the level l. This implies the mutex works for $l = n-1$.
The algorithm has a starvation freedom property such that every process that invokes lock and is at level $l \leq n-1$ eventually wins and enters the CS.

Proof of safety:
Proof by induction on the level l:

1. Base case: $l = 0$
2. Induction: a process p can increase its level by writing its `FLAG` at $l+1$ and its index in `LAST[l+1]`. Let p_x be the last one that writes `LAST[l+1]=x`, for p_x to pass at level $l+1$, it must be that $\forall k$ `FLAG[k]<l+1`. So, p_x is the only proc at level $l+1$.

   Otherwise, p_x is blocked in the wait and so we have at most $n-l-1$ processes at level $l+1$, those at level l, that by induction are at most $n-l$, except for p_x that is blocked in its $(l+1)$-th wait.

Proof of liveness:
Proof by reverse induction on the level l:

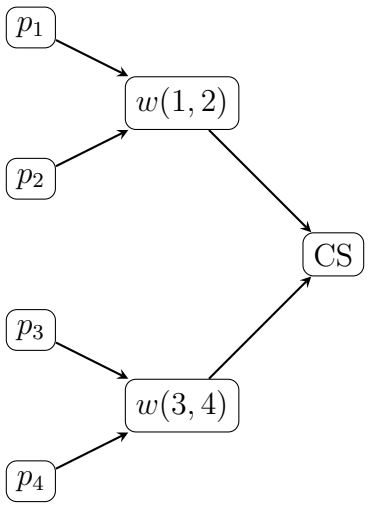
1. Base case: $l = n - 1$
2. Induction: assume a p_x is blocked at level l, so $\exists k$ `FLAG[k]`$\geq$`l+1` $\wedge$ `LAST[l+1]=x`. If some p_y sets `LAST[l+1]=y` p_x will exit from its wait and pass to level $l + 1$.

   Otherwise, let $G = \{p_i |$ `FLAG[i]` $\geq$ `l+1`$\}$ and $L = \{p_i |$`FLAG[i]<l+1`$\}$. Every $p \in L$ will never enter its $(l + 1)$-th loop and every $p \in G$ will eventually win and move into L. Eventually, p_x will be the only one in its $(l + 1)$-th loop, with all other processes at level $l' < l + 1$ and p_x will eventually pass to level $l + 1$ and win.

The algorithm doesn't satisfy bounded bypass because if a process is sleeping the other can enter the CS an unbounded number of times.

1.3.2 Tournament based algorithm

To reduce the cost of a mutex we can use a tournament betwen pairs of processes:



For the competition between two processes the Peterson algorithm is used and this means that a process enters the CS after $\lceil \log 2 \rceil$ competitions, each of constant cost for a total cost of $O(\log n)$. This algorithm has also a bounded bypass with bound $\lceil \log n \rceil$.

1.3.3 Fast mutex algorithm

Is it possible to reduce the cost to $O(1)$, using the **Fast mutex** algorithm by Lamport:

Algorithm: Fast mutex

```
// Inizialize Y as ⊥, X as 0
def lock(i):
    FLAG[i]=true // *
    X=i
    if Y!=⊥ :
        FLAG[i]=false
        wait Y = ⊥
        goto *
    else :
        Y=i
        if X=i :
            return
        else :
            FLAG[i]=false
            wait ∀j FLAG[j]=false
            if Y=i :
                return
            else :
                wait Y=⊥
                goto *

def unlock(i):
    Y=⊥
    FLAG[i] = 0
    return
```

This algorithm require 5 accesses to the registers if there is no contention.

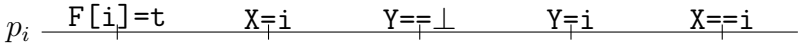
Fast mutex properties

The algorithm designs a correct mutex with the deadlock freedom property.

Proof of safety:

Proof by contraddiction, assuming two processes p_i and p_j are inside the CS. How can p_i enter the CS:

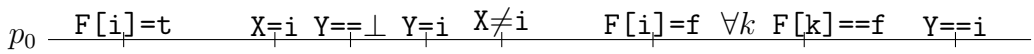
1. In the first if `X=i`:



   For p_j to enter the CS, it must read `Y=`$\perp$, so it must read `Y` before p_i writes it equal to i. Also p_j must write `X=j` and it must have done it before p_i writes it equal to i, otherwise

p_i couldn't have read the value of `X` equal to i. So, when p_j reads `X`, it finds it different from j and cannot enter the CS.

2. In the second if `Y=i`:



For p_j to enter the CS, it must read `Y=`$\perp$, so it must read `Y` before p_i writes it equal to i. Also p_j must write `X=j`, if it has done it before p_i writes it equal to i, then is the same case as before. If p_j writes `X=j` after and finds it at j then it enters its CS and p_i is blocked. If p_j doesn't find `X` equal to j then it must have written `Y` before p_i and is blocked until p_i finishes the CS.

Proof of liveness:
Let p_i invock lock, where it can be blocked forever:

1. In the second wait `Y=`$\perp$: In this case there is a process p_j that wrote `Y=j` the last and it will eventually win, freeing p_i.

2. In the wait $\forall k$ `FLAG[k]=false`: This wait cannot block a process forever because every process that doesn't enter the CS puts its flag down and the one that enters the CS puts its falg down after exiting.

3. The first wait `Y=`$\perp$: In this case there is a process p_j that wrote `Y=j` before and it must eventually win because unless it must be blocked as well, but we already proved that this is not possible.

1.3.4 Deadlock freedom to Bounded bypass

Let DFL be a deadlock free protocol for mutex, we can turn it in a bounded bypass protocol using the **Round Robin** algorithm:

Algorithm: Round Robin

```
// Inizialize FLAG[i]=false, TURN as a random process id
def lock(i):
    FLAG[i]=true
    wait (TURN=i OR FLAG[TURN]=false)
    DFL.lock(i)
    return

def unlock(i):
    FLAG[i] = 0
    if FLAG[TURN]=false :
        TURN=(TURN+1)% n
    DFL.unlock(i)
    return
```

Deadlock freedom of RR

If at least one process invokes lock, then at least one process enters the CS.

Proof:

The only way to that being false is if every process is locked in the wait before invocking `DFL.lock(i)`. If `TURN=k` and p_k invokes the lock then it exit the wait, otherwise its flag is down and the other processes can exit their wait.

Maximum number of iterations

If `TURN=i` and `FLAG[i]=true`, then p_i enters the CS in at most $n-1$ iterations.

Proof:

The turn changes only when `FLAG[i]` is down, if `FLAG[i]` is up it means that p_i is in its CS or p_i is competing for its CS and will eventually invokes `DLF.lock`. If p_j invokes lock after that `FLAG[i]` is set, p_j blocks in its wait.

Let Y be the set of processes competing for the CS (suspended on `DLF.lock`). We have that $i \in Y$ and Y can't grow anymore. DLF is deadlock free so eventually some process must win and exit from Y, in the worst case all process are in Y and p_i enters the CS after all of them ($n-1$ processes).

Maximum number of turns

If `FLAG[i]` is up, then `TURN` is set to i in at most $(n-1)^2$ iterations.

Proof:

Given a random process p_i that invokes lock, when `FLAG[i]` is set the turn is of another process p_j. By the previous lemma p_j wins after at most $n-1$ iterations and the turn is increased. The worst case scenario is if the the index of p_j is equal to $i+1$, so p_i needs to wait at most $(n-1)$ turns, each with at most $(n-1)$ iterations, for a total of $(n-1)^2$ iterations. This means that in total the bound of this algorithm is $n(n-1)$.

1.4 Special HW primitives

Operating systems usually provides specialized HW primitives, that perform in an atomic way the combination of some instructions. Using this primitives is very easy to solve the mutex problem.

1.4.1 Test&Set

Let X be a boolean register, the **Test&Set** read the register and sets it to 1. Is implemented as follows:

Algorithm: Test&Set

```
def X.Test&Set():
    tmp = X
    X = 1
    return tmp
```

Using this algorithm the mutex can be implemented using a simple protocol:

Algorithm: Mutex with Test&Set

```
// Inizialize X at 0
def lock(i):
    wait (X.Test&Set = 0)
    return

def unlock(i):
    X = 0
    return
```

1.4.2 Swap

Let X be a register, the **Swap** read the register and sets it to an input value v. Is implemented as follows:

Algorithm: Swap

```
def X.Swap(v):
    tmp = X
    X = v
    return tmp
```

Using this algorithm the mutex can be implemented using a simple protocol:

Algorithm: Mutex with Swap

```
// Inizialize X at 0
def lock(i):
    wait (X.Swap(1) = 0)
    return

def unlock(i):
    X = 0
    return
```

1.4.3 Compare&Swap

Let X be a register, the **Compare&Swap** check if the register is equal to an input value *old* and sets it to an input value *new*. Is implemented as follows:

Algorithm: Compare&Swap

```
def X.Compare&Swap(old, new):
    if X = old :
        X = new
        return true
    return false
```

Using this algorithm the mutex can be implemented using a simple protocol:

Algorithm: Mutex with Compare&Swap

```
// Inizialize X at 0
def lock(i):
    wait (X.Compare&Swap(0, 1) = True)
    return

def unlock(i):
    X = 0
    return
```

1.4.4 Fetch&Add

Let X be a register, the **Fetch&Add** read the register and to it an input value v. Is implemented as follows:

Algorithm: Fetch&Add

```
def X.Fetch&Add(v):
    tmp = X
    X = X+v
    return tmp
```

Using this algorithm the mutex can be implemented using a simple protocol:

Algorithm: Mutex with Swap

```
// Inizialize TICKET and NEXT at 0
def lock(i):
    my_tick = TICKET.Fetch&Add(1)
    wait (my_tick = NEXT)
    return

def unlock(i):
    NEXT = NEXT+1
    return
```

1.5 Safe registers

Atomic Read/Write registers and HW primitives provide a form of atomicity. From now we will consider registers with no atomicity, the Safe registers. A safe register provides read and write in such a way:

- Every read that does not overlap with a write returns the value stored in the register.
- A read that overlaps with a write returns any value.
- In a MRMW register when two writes overlap the register can contain any value.

This is the weakest type of registers useful for concurrency.

1.5.1 Bakery algorithm

A simple protocol that works with safe registers is the Bakery algorithm by Lamport:

Algorithm: Bakery algorithm

```
// Inizialize FLAG[i] = false and MY_TURN[i] to 0
def lock(i):
    // Start doorway
    FLAG[i] = true
    MY_TURN[i] = max(MY_TURN[0],...,MY_TURN[n])+1
    FLAG[i] = false
    // End doorway
    // Start bakery
    for j≠i :
        wait FLAG[j] = false
        wait (MY_TURN[j] = 0 or [MY_TURN[i],i] < [MY_TURN[j],j])
    // End bakery
    return

def unlock(i):
    MY_TURN[i] = 0
    return
```

Turn order

Let p_i be in the CS and p_j in the doorway or the bakery, then:

$$[\texttt{MY\_TURN[i]},\texttt{i}] < [\texttt{MY\_TURN[j]},\texttt{j}]$$

This means that the mutex works as intended.
Also the algorithm has the bounded bypass property with bound $n - 1$.

Proof of safety:
If p_i is in the CS it has passed the waits for j, the read of `FLAG[j]` done by p_i in respect to the execution of p_j can be:

- The read overlaps with `FLAG[j] = t`, then p_j will read the value written in `MY_TURN[i]` and will take an higher number.
- The read overlaps with p_j being in the bakery, so the value for `MY_TURN[j]` has been set and p_i has evaluated that [`MY_TURN[i]`,i]<[`MY_TURN[j]`,j].

Proof of liveness:
By contraddiction, assume that there is a lock but no process enters the CS. Where all the process are blocked:

- The doorway cannot block a process forever, so the first wait cannot block forever aswell.
- For the second wait the turn are totally ordered, so there is a process p_i for which the turn is the minimum that will enter the CS.

The ticket ordered means that a process cannot surpass another process 2 times, so at most a process could wait $n-1$ processes before entering the CS.

1.5.2 Aravind algorithm

In the Lamport algorithm the registers are unbounded and this could be a problem. So, another algorithm was proposed with a reset method for the tickets:

Algorithm: Aravind algorithm

```
// Inizialize FLAG[i] = false, STAGE[i] to 0 and DATE[i] to i
def lock(i):
    FLAG[i] = true
    do:
        STAGE[i] = 0
        wait (∀j ≠ i FLAG[j] = false or DATE[i] < DATE[j])
        STAGE[i] = 1
    while ∀j ≠ i STAGE[i]=0
    return

def unlock(i):
    tmp = max(DATE)+1
    if tmp ≥ 2n :
        for ∀ j :
            DATE[j] = j
    else :
        DATE[i] = tmp
    STAGE[i] = 0
    FLAG[i] = false
    return
```

Aravind algorithm properties

Two processes cannot be in the CS simultaneously, ensuring that the mutex works. Also the algorithm has the bounded bypass property with bound $2n-2$.

Proof of safety:

Let us consider the execution of p_i leading to its CS. There is another process p_j for which p_i has done the last read of `STAGE[j]=0` before entering the CS. To enter the CS p_j has to write `STAGE[j]=1`, so its write must overlap with the read of p_i. Then p_j will read `STAGE[i]` and found it at 1, so it will not enter the CS.

Proof of liveness:

Let p_i invoke the lock and let us consider the moment after the reset of `DATE` occurs. Another process p_j that enters the CS before p_i cannot surpass it again because `DATE[j]` will be higher than `DATE[i]`. In the worst case p_i could be the n-th process to enter the CS after the reset, but cannot experience two resets during its lock.

If p_n invokes the lock alone and sets `DATE[i]` to $n+1$ and then all process lock simultaneously, there will be a reset done by p_{n-1} after p_n has waited $n-1$ processes. After the reset it will wait the other $n-1$ processes to enter the CS (because it has the higher `DATE` after the reset), making the bound $2n-2$.

The algorithm could be revisioned changing the unlock function as follows:

Algorithm: Aravind unlock revisioned

```
def unlock(i):
    for ∀ j :
        if DATE[j]>DATE[i] :
            DATE[j] = DATE[j]-1
    DATE[i] = n
    STAGE[i] = 0
    FLAG[i] = false
    return
```

This new version lowers the bound to $n-1$.

1.6 Semaphores

1.6.1 Semaphor definition

A semaphor is a shared counter S, accessed with two primitives **up** and **down** such that:

1. S is inizialized at a value $s_0 \geq 0$
2. S is always ≥ 0
3. The primitive **up** atomically increases S
4. The primitive **down** atomically decreases S if it's not 0, otherwise the process calling the primitive is blocked

The semaphores are made of a counter, inizialized at s_0 that can become negative. Stated that the value of the semaphor cannot be negative, when the value is positive represents the value of the semaphor otherwise counts how many process are suspended. The semaphor has also a queue to store suspended processes.
The implementation of a semaphor, defining the two primitives is:

Algorithm: Semaphor

```
// Inizialize t to 0
def S.down():
    Disable interrupts
    wait S.t.test&set()=0
    S.counter--
    if S.counter<0 :
        Enter into S.queue
        S.t=0
        Enable interrupts
        SUSPEND
    else :
        S.t=0
        Enable interrupts
    return

def S.up():
    Disable interrupts
    wait S.t.test&set()=0
    S.counter++
    if S.counter≤0 :
        Activate process in S.queue
    S.t=0
    Enable interrupts
    return
```

1.6.2 Example: Producer-Customer

We have a shared FIFO buffer of size k and two function, one to insert (produce) a value and one to remove (consume) a value.
In the internal representation we have:

1. A set of register BUF[k] not even safe accessed in Mutex.
2. Two semaphores FREE and BUSY that count the number of free cells in the buffer, inizialized at k and 0.
3. Two arrays FULL ad EMPTY of atomic boolean registers to track which cell are empty, inizialized to false and true.
4. Two semaphores SP and SC, inizialized at 1.

The implementation of the two functions with semaphores is:

Algorithm: Producer-Customer

```
def B.produce(v):
    FREE.down()
    SP.down()
    while not EMPTY[IN] :
        IN = (IN+1)% k
    i = IN
    EMPTY[IN] = false
    SP.up()
    BUF[i] = v
    FULL[i] = true
    BUSY.up()
    return

def B.consume():
    BUSY.down()
    SC.down()
    while FULL[OUT] :
        OUT = (OUT+1)% k
    o = OUT
    FULL[OUT] = false
    SC.up()
    tmp = BUF[o]
    EMPTY[o] = true
    FREE.up()
    return tmp
```

1.6.3 Example: Reader-Writer

We have several processes that want to access a file, with reader processes that can access simultaneously and writer that must be one at time. Also readers and writers are mutually exclusive.

The operations will have this shape:

Algorithm: Reader-Writer

```
def conc_read():
    begin_read()
    read()
    end_read()

def conc_write():
    begin_write()
    write()
    end_write()
```

Based on the priority that we want readers and writers to have different implementation of the read and write function can be done.

The first case is with weak priority to readers, means that if a reader arrives during a read it surpass the writers suspended. When a writer terminates it activate the first suspended process, irrispectively of wheter it is a reader or a writer.

Algorithm: Reader-Writer (Weak priority to readers)

```
// GLOB_MUTEX and R_MUTEX initialized to 1, R inizialized to 0.
def begin_read():
    R_MUTEX.down()
    R++
    if R=1 :
        GLOB_MUTEX.down()
    R_MUTEX.up()
    return

def end_read():
    R_MUTEX.down()
    R--
    if R=0 :
        GLOB_MUTEX.up()
    R_MUTEX.up()
    return


def begin_write():
    GLOB_MUTEX.down()
    return

def end_write():
    GLOB_MUTEX.up()
    return
```

We could also give strong priority to readers, means that when a writer terminates it activates the first reader and only if there aren't readers it activates the writer.

Algorithm: Reader-Writer (Strong priority to readers)

```
// GLOB_MUTEX, R_MUTEX and W_MUTEX initialized to 1, R inizialized to 0.
def begin_read():
    // Same as before

def end_read():
    // Same as before


def begin_write():
    W_MUTEX.down()
    GLOB_MUTEX.down()
    return

def end_write():
    GLOB_MUTEX.up()
    W_MUTEX.up()
    return
```

The last case is with weak priority to writers, means that when a writer terminates it activates the first writer and only if there aren't writers it activates the reader.

Algorithm: Reader-Writer (Weak priority to writers)

```
// All MUTEX initialized to 1, R inizialized to 0.
def begin_read():
    PRIO_MUTEX.down()
    R_MUTEX.down()
    R++
    if R=1 :
        GLOB_MUTEX.down()
    R_MUTEX.up()
    PRIO_MUTEX.up()
    return

def end_read():
    // Same as before


def begin_write():
    W_MUTEX.down()
    W++
    if W=1 :
        PRIO_MUTEX.down()
    W_MUTEX.up()
    GLOB_MUTEX.down()
    return

def end_write():
    GLOB_MUTEX.up()
    W_MUTEXdown()
    W--
    if W=0 :
        PRIO_MUTEX.up()
    W_MUTEX.up()
    return
```

1.7 Monitors

Monitors are concurrent objects that guarantee that at most one operation invocation is active inside it at the same time. The inter-process organization is done by conditions, objects that provide two operations:

- wait: the invocking process suspends, releasing the mutex on the monitor.
- signal: based on the semantics that we want to use there are two possible implementations:
  - Hoare semantics: reactivates the first process suspended and suspends the invocking process, that however has priority to reenter.
  - Mesa semantics: the invocking process completes its task and the first process suspended has the priority to enter after.

  In both cases if there are no process suspended nothing happens.

The implementation of a monitor with Hoare semantics is done by using semaphors, in particular:

- A semaphor MUTEX that guarantee mutex for the monitor.
- A semaphor PRIO and a value N_{PR} to count the number of process that have priority.
- For every condition C a semaphor SEM_C and a value N_C to count the number of suspended process for the condition.

Every monitor operation start with `MUTEX.down()` and ends with:
`if` N_{PR}`>0: PRIO.up() else: MUTEX.up()`
The two operation of a condition are implemented as follows:

Algorithm: Condition

```
// SEM_C, PRIO, N_C, N_PR initialized to 0 and MUTEX inizialized to 1.
def C.wait():
    N_C++
    if N_PR>0 :
        PRIO.up()
    else :
        MUTEX.up()
    SEM_C.down()
    N_C--
    return

def C.signal():
    if N_C>0 :
        N_PR++
        SEM_C.up()
        PRIO.down()
        N_PR--
    return
```

1.7.1 Example: Barrier

Randez-vous is a concurrent object associated with m control points, one for every process, each that can be passed when all processes are in their control points. The set of all control points is a barrier.

Algorithm: Barrier with monitor

```
def monitor RNDV:
    c=0
    condition B

    def barrier():
        c++
        if c<m :
            B.wait()
        else :
            c=0
        B.signal()
        return
```

1.7.2 Example: Reader-Writer

We can rewrite the reader-writer problem in a simpler way with a monitor. The case of strong priority to readers is now implemented as follows:

Algorithm: Reader-Writer (Strong priority to readers)

```
def monitor RW_READERS:
    AR, WR, AW, WW = 0, 0, 0, 0
    condition CR, CW

    def begin_read():
        WR++
        if AW≠0 :
            CR.wait()
            CR.signal()
        AR++
        WR--

    def end_read():
        AR--
        if AR+WR=0 :
            CW.signal()


    def begin_write():
        if AR+WR≠0 or AW≠0 :
            CW.wait()
        AW++

    def end_write():
        AW--
        if WR>0 :
            CR.signal()
        else :
            CW.signal()
```

We can also now implement a case with strong priority to writers:

Algorithm: Reader-Writer (Strong priority to writers)

```
def monitor RW_WRITERS:
    AR, WR, AW, WW = 0, 0, 0, 0
    condition CR, CW

    def begin_read():
        if WW+AW≠0 :
            CR.wait()
            CR.signal()
        AR++

    def end_read():
        AR--
        if AR=0 :
            CW.signal()


    def begin_write():
        WW++
        if AR+AW≠0 :
            CW.wait()
        AW++
        WW--

    def end_write():
        AW--
        if WW>0 :
            CW.signal()
        else :
            CR.signal()
```

Both this solutions suffer from starvation of one of the two type of processes, readers or writers. So, we can implement a fair solution to solve this problem:

Algorithm: Reader-Writer (Fair)

```
def monitor RW_FAIR:
    AR, WR, AW, WW = 0, 0, 0, 0
    condition CR, CW

    def begin_read():
        WR++
        if WW+AW≠0 :
            CR.wait()
            CR.signal()
        AR++
        WR--

    def end_read():
        AR--
        if AR=0 :
            CW.signal()


    def begin_write():
        WW++
        if AR+AW≠0 :
            CW.wait()
        AW++
        WW--

    def end_write():
        AW--
        if WR>0 :
            CR.signal()
        else :
            CW.signal()
```

1.7.3 Dining Philosopher problem

The dining philosopher problem is a famous problem of concurrency in which there are n philosopher seated around a circular table with a fork in between each pair of philosophers. To eat a philosopher must take the two nearest fork and it can't pick up both simultaneously, but only one at a time.
To solve this problem with a deadlock-free solution there are different methods. Some solutions break the simmetry of the system:

1. All philosophers first pick the right fork except one that first pick the left one.
2. Odd philosophers pick the left first and even philosophers pick the right first.
3. Allow at most $n-1$ philosophers with n forks.

Another solution where simmetry is not broken is to check if both forks are free before taking them. This can be implemented through a monitor.
The solution follows this steps:

- A philosopher moves into eating state only if his neighbors are both not in their eating state (the fork are both free).
- If a neighbor of a philosopher is eating, it will signal when it finishes (the fork is again free).

This solution is perfect to be implemented with monitors:

Algorithm: Dining Philosophers

```
def monitor DP:
    state[n] = thinking // for each n
    condition self[n] // n conditions

    def Pickup(i):
        state[i] = hungry
        test(i)
        if state[i] != eating :
            self[i].wait()

    def Putdown(i):
        state[i] = thinking
        test((i+1)%n)
        test((i-1)%n)

    def test(i):
        if state[(i+1)%n]!=eating and state[(i-1)%n]!=eating and
         state[i]=hungry :
            state[i] = eating
            self[i].signal()
```

1.8 Software Transactional Memory

Software Transactional Memory groups parts of the code that must look like atomic and easy to use for the programmer. The part of the code to group is not defined by the object but is application dependent.

Transaction

A transaction is an atomic unit of computation that can access atomic objects and doesn't overlap with other transactions.

An STM system is an online algorithm that ensures that the transactional code of the program is executed as atomic.
Different transaction are executed simultaneously, with the **optimistic execution** approach, to guarantee efficiency. Each transaction is composed by 3 parts:

1. Read of atomic registers
2. Local computation
3. Write into shared memory

The shared memory must remain consistent and when an inconsistency is discovered, the transaction is aborted.

1.8.1 Logical clock based STM system

For a transaction T the read prefix is the set of all the successful read before its possible abortion. An execution is **opaque** if there is a total order between committed transaction and read prefixes of aborted transaction.
An atomic STM system is the **Transactional Locking 2**, in which there are:

- CLOCK is a READ/FETCH&ADD register
- Every MRMW register is implemented as pair XX, where:
  - XX.val contains the value of X
  - XX.date contains the date of last update in terms of CLOCK
- For every transaction T, the invocking process has:
  - lc(XX), a local copy of the register X
  - read_set(T), the set of all register read by T
  - write_set(T), the set of all registers written by T
  - birthdate(T), the value of CLOCK at the start of T

We want to commit a transaction as if it was all executed at its birthdate and is consistent. So, every register read and written must be modified before the transaction starts.

The implementation of this system through four functions is:

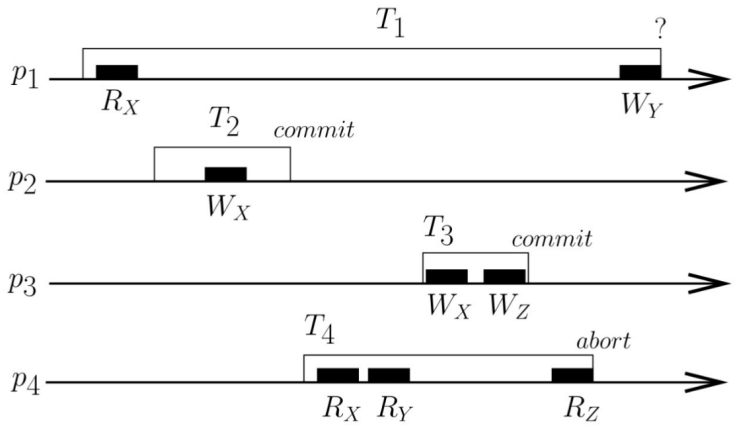
Algorithm: Logical Clock system

```
def begin_T():
    read_set(T) = {}
    write_set(T) = {}
    birthdate = CLOCK+1

def X.read_T():
    if lc(XX)≠None :
        return lc(XX).val
    lc(XX) = XX
    if lc(XX).date ≥ birthdate(T) :
        ABORT
    read_set(T).add(X)
    return lc(XX).val

def X.write_T(v):
    lc(XX).val = v
    write_set(T).add(X)

def try_to_commit_T():
    // lock all registers in read_set(T) and write_set(T)
    for X in read_set(T) :
        if XX.date≥birthdate(T) :
            // release all locks
            ABORT
    tmp = CLOCK.fetch&add(1)+1
    for X in write_set(T) :
        XX = (lc(XX).val, tmp)
    // release all locks
    COMMIT
```

For every register the first read copies the values from the register and the successive ones read it from the local copy. For the write they all modify the local copy and the value is written only at the end of the transaction.

1.8.2 Vector clock based STM system

The opacity property used in the logical clock system is a very strict property that could abort a transaction based on a already aborted one. For example in this case:



T4 must be aborted because it read Z after it is modified by T3 and X before it is modified by T3, so it cannot be placed before or after. With the opacity property the read prefix of T4 (X,Y) also cannot be placed in any point in the middle of T1 because reads X modified by T1 but Y before the write of T1, so T1 is aborted and an aborted transaction (T4) lead T1 to abort.

We define a less strict property called **Virtual World Consistency** in which the committed transaction have a total order like before, but aborted transaction read prefixes have a partial order with committed transaction.

With this property in the example before the transaction would be committed with order T1<T2<T3 and there would be a partial order T2<read_p(T4).

An SMT system with the VWC property is the vector clock system, in which:

- Every MRMW register is implemented as a pair XX, with:
  - XX.val contains the value of X
  - XX.depend[m] is a vector clock such that:
    - XX.depend[X] is the sequence number associated with the current value of X
    - XX.depend[Y] is the sequence number associated with the value of Y from which the current value of X depends
- Every process p_i has:
  - For every register X a local copy lc(XX)
  - p_i_depend[m] a vector of all sequence number such that p_i_depend[X] is the last value of X that p_i knows
- As before every transaction has a read_set and a write_set but also:
  - t_depend[m], a local copy of p_i_depend[m] to not change it if the transaction is aborted

The sequence number is the date of update of X in relation to the vector clock.
The implementation of the 4 functions is:

Algorithm: Vector Clock system

```
def begin_T():
    read_set(T) = {}
    write_set(T) = {}
    t_depend = p_i_depend

def X.read_T():
    if lc(XX)=None :
        lc(XX) = X
        read_set(T).add(X)
        t_depend[X] = lc(XX).depend[X]
        if ∃ Y ∈ read_set(T)| t_depend[Y]<lc(XX).depend[Y] :
            ABORT
        for Y not in read_set(T) :
            t_depend[Y] = max(t_depend[Y], lc(XX).depend[Y])
    return lc(XX)

def X.write_T(v):
    lc(XX).val = v
    write_set(T).add(X)

def try_to_commit_T():
    // lock all registers in read_set(T) and write_set(T)
    if ∃ Y ∈ read_set(T)|t_depend[Y]<YY.depend[Y] :
        // release all locks
        ABORT
    for X in read_set(T) :
        t_depend[X] = XX.depend[X]+1
    for X in write_set(T) :
        XX = (lc(XX).val, t_depend)
    // release all locks
    p_i_depend = t_depend
    COMMIT
```

1.9 Linearizability

We have a set of processes $p_1, \ldots, p_n$ that access a set of concurrent objects $X_1, \ldots, X_m$ by invoking operations in the form `X_i.op(args)(ret)`. An operation invocked is modeled by two events `inv[X_i.op(args)]` and `res[X_i.op(ret)]`.

History

A history (or trace) is a pair $\hat{H} = (H, <_H)$ where H is a set of events and $<_H$ is the total order of the events such that every res follows the corresponding inv.

An history is sequential if every res is the return operation of the previous inv. Its complete if every inv has a res.

Linearizability

A complete history $\hat{H}$ is linearizable if exists a sequential history $\hat{S}$ such that:

1. $\forall X\ \hat{S}|_X \in \text{semantics}(X)$

   The semantics means how the object works, for example in a queue you must enqueue an item before dequeing it.

2. $\forall p\ \hat{H}|_p = \hat{S}|_p$

   The code of a process must be executed in the correct order.

3. If res[op]$<_H$inv[op'] $\implies$ res[op]$<_S$inv[op']

   It means only overlapping operations can be rearranged.

Given an history $\hat{H}$ we define a binary relation $\rightarrow_H$ such that:

$$\text{op} \rightarrow_H \text{op'} \iff \text{res[op]} <_H \text{inv[op']}$$

So, for the condition 3 we have that $\rightarrow_H \subseteq \rightarrow_S$.

1.9.1 Compositionality

Compositionality

An history $\hat{H}$ is linearizable if $\hat{H}|_X$ is linearizable for every X involved in $\hat{H}$.

Proof:
Let $\hat{S}_X$ be the linearization of $\hat{H}|_X$ with the total order $\rightarrow_X$.
Let $\rightarrow$ be the set of pairs created by:

$$\rightarrow = \rightarrow_H \cup \bigcup_{\forall X \in \hat{H}} \rightarrow_X$$

The relation $\rightarrow$ is acyclic, because:

1. Cannot have cycle of 1 edge, because it means that res[op]<inv[op] that is impossible.
2. Cannot have cycles of 2 edges, op$\rightarrow$op'$\rightarrow$op, because:
   - The two relation cannot be of the same type ($\rightarrow_H$ or $\rightarrow_X$) otherwise the relation would be cyclic in the beginning.
   - The relation cannot also be on two different object op$\rightarrow_X$op'$\rightarrow_Y$op otherwise an operation would be on two objects at the same time.
   - The two relation must be op$\rightarrow_X$op'$\rightarrow_H$op (or vice versa) and op'$\rightarrow_H$op implies that res[op']$<_H$inv[op] but $\hat{S}_X$ is the linearization of $\hat{H}|_X$ so res[op']$<_X$inv[op]. So, the relation op$\rightarrow_X$op' cannot exists if $\rightarrow_X$ is acyclic.

3. Cannot have cycles of more than 2 edges, let consider the shortest cycle, that has the form:

$$\text{op1} \rightarrow_H \text{op2} \rightarrow_X \text{op3} \rightarrow_H \text{op4}$$

This means that:

- The relation op1$\rightarrow_H$op2 means that res[op1]$<_H$inv[op2].
- The relation op2$\rightarrow_X$op3 implies inv[op2]$<_H$res[op3] (they overlap or op2 is before op3).
- The relation op3$\rightarrow_H$op4 means res[op3]$<_H$inv[op4].

By transitivity on $<_H$ we have that res[op1]$<_H$inv[op4] then op1$\rightarrow_H$op4 and by contradiction this is not the shortest cycle.

Topological order of $\rightarrow$

The relation $\rightarrow$ is an acyclic graph, so it admits a topological order $\rightarrow'$. Then exists $\hat{S}$, the linearization of $\hat{H}$ defined as follows:

$$\hat{S} = \text{inv(op1)res(op1)inv(op2)res(op2)}\ldots \qquad \text{where op1} \rightarrow' \text{op2} \rightarrow' \ldots$$

Proof:
We need to prove the 3 property of the linearization:

1. $\forall X\ \hat{S}|_X = \hat{S}_X \in \text{semantics}(X)$:

   We have that:

   $$<_{\hat{S}_X} = \rightarrow_X \subseteq \rightarrow\ |_X \subseteq \rightarrow'\ |_X = \rightarrow_{\hat{S}_X} = <_{\hat{S}|_X}$$

   The two orders $<_{\hat{S}_X}$ and $<_{\hat{S}|_X}$ are total orders on the same events, so they must coincide.

2. $\forall p\ \hat{H}|_p = \hat{S}|_p$:

   A process is sequential so:

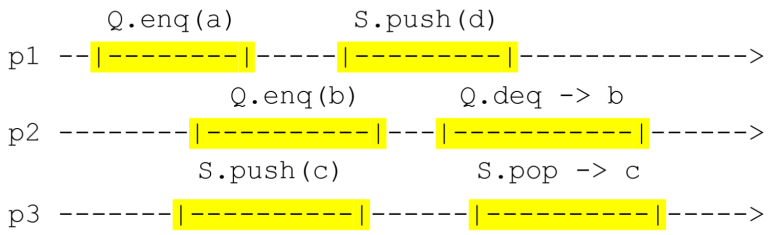
   $$\hat{H}|_p = \text{inv(op1)res(op1)} \cdots = \hat{S}|_p$$

   because op1$\rightarrow_H$op2$\rightarrow_H \ldots$ and $\rightarrow_H \subseteq \rightarrow'$.

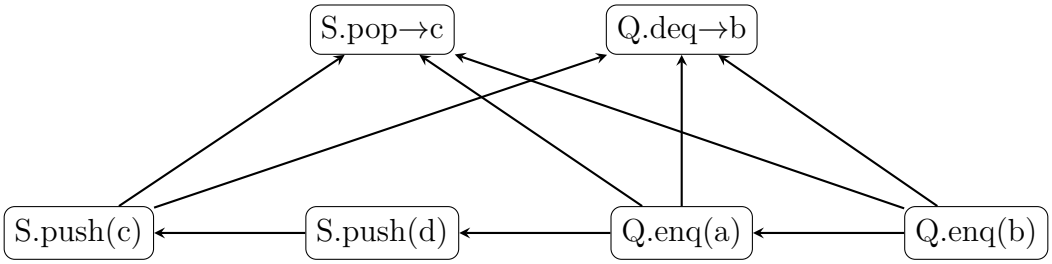
3. If res[op]$<_H$inv[op'] $\implies$ res[op]$<_S$inv[op']:

   By definition $\rightarrow_H \subseteq \rightarrow \subseteq \rightarrow' = \rightarrow_S$.

Example:
Given this execution:



This execution generates this graph:



That has two possible linearizations:

1. Q.enq(b), Q.enq(a), S.push(c), S.push(d), Q.deq→b, S.pop→c
2. Q.enq(b), Q.enq(a), S.push(c), S.push(d), S.pop→c, Q.deq→b

1.9.2 Sequential consistency

We define the relation op→$_{proc}$op' such that if a process invockes both op and op' we have that res[op]<inv[op'].

Sequential consistency

A complete history $\hat{H}$ is sequential consistent if exists a sequential history $\hat{S}$ such that:

1. $\forall X\ \hat{S}|_X \in \text{semantics}(X)$
2. $\forall p\ \hat{H}|_p = \hat{S}|_p$
3. $\rightarrow_{proc} \subseteq \rightarrow_S$

This property is a more generous notion than linearizability.
Sequential consistency however is not compositional, for example with two processes:

1. Q.enq(a), Q'.enq(b'), Q'.deq→b'
2. Q'.enq(a'), Q.enq(b), Q.deq→b

The two processes are both sequentially consistent but there is no way to put them together and have an history sequentially consistent.

1.9.3 Serializability

Instead of processes we consider transactions (defining $\rightarrow_{trans}$ in the same way as $\rightarrow_{proc}$), where each transaction t has either abort(t) or commit(t) as event. A sequential history is formed by committed transactions only.

Serializability

A complete history $\hat{H}$ is serializable if exists a sequential history $\hat{S}$ such that:

1. $\forall X \ \hat{S}|_X \in \text{semantics}(X)$
2. $S = \{e \in H | e \in t \in \text{committedT}(\hat{H})\}$
3. $\rightarrow_{trans} \subseteq \rightarrow_S$

Like sequential consistency this property is more generous than linearizability but is not compositional.

2

Concurrency without Mutex

2.1 Mutex-free property

The critical section used in mutual exclusion have drawbacks, like having to choose the right level of granularity and the fact that a crash during a CS could block the entire system.
With a Mutex-free approach the only atomicity provided is the atomicity of primitives.
We have to define new liveness properties, because the old ones rely on CS:

1. **Obstruction freedom**: every time an operation on an object is run in isolation, it terminates.
2. **Non-blocking**: when an operation is invoked on an object, eventually one operation on that object terminates.
3. **Wait freedom**: whenever an operation is invocked, it eventually terminates.
4. **Bounded wait freedom**: wait freedom with a bound on the number of steps needed to terminate.

All this properties cope with crashes and there is no way in an asynchronous system to distinguish a failure from an arbitrary long sleep.

2.2 Example of mutex-free

2.2.1 Splitter

A splitter is a concurrent object that provide a single operation `dir` such that:

1. Validity: it returns L, R or S (left, right or stop).
2. Concurrency: with n simultaneous invocations of `dir`:
   - At most $n-1$ return L
   - At most $n-1$ return R
   - At most 1 return S
3. Wait freedom: it eventually terminates.

To implement the splitter we use:

- DOOR: MRMW boolean register
- LAST: MRMW register

Algorithm: Splitter

```
// DOOR initialized at 1 and LAST to a random process id
def dir(i):
    LAST=i
    if DOOR=0 :
        return R
    else :
        DOOR=0
        if LAST=i :
            return S
        else :
            return L
```

Soundness of splitter

The implemntation satisfied the 3 requirement fo the splitter.

Proof:
The validity and wait freedom are trivial.
For concurrency we have that:

- Not all process can obtain R because the process that sets `DOOR=0` cannot obtain R.
- Not all process can obtain L because the last process that writes `LAST=i`, even if it finds the `DOOR` to 1, it cannot recieve L.
- For the S, let p_i be the first process that recieves S, so `LAST=i`, so no other process p_j have written it between the writing and reading of p_i. So, the other process can't find `LAST=j` and the door to 1, not recieving S.

2.2.2 Timestamp generator

A timestamp generator is a concurrent object with an operation `get_ts` such that:

1. Validity: every invocation returns different values.
2. Concurrency: if one process terminates the invocation of `get_ts` before another process starts it, then the first process has a timestamp smaller than the second one.
3. Obstruction freedom: it eventually terminates if run in isolation.

To implement the timestamp generator we use:

- DOOR[i]: MRMW boolean registers
- LAST[i]: MRMW registers
- NEXT: integer

Algorithm: Timestamp generator

```
// DOOR[i] initialized at 1, LAST[i] to a random process id and NEXT
inizialized to 1
def get_ts(i):
    k=NEXT
    while true :
        LAST[k]=i
        if DOOR[k]=1 :
            DOOR[k]=0
            if LAST[k]=1 :
                NEXT++
                return k
        k++
```

This timestamp generator suffer from deadlock if every time a process p_i set `DOOR[k]=0` it doesn't find `LAST[k]=i` because another process p_j has written it. The process p_j also cannot obtain its timestamp because the door is set to 0.

Soundness of timestamp generator

The implementation satisfied the 3 requirement for the timestamp generator.

Proof:
Validity and obstruction freedom are trivial.
For consistency, a process increase the value of NEXT when it terminates so every process that starts after it finds NEXT to an higher value and its value never decreases.

2.2.3 Stack

To implement a stack we use:

- REG[i]: an unbounded array of atomic registers that can be:
  - Written
  - Read with `swap(v)` primitive
- NEXT: register that points the next free location of the stack, that can be:
  - Read
  - Written with `fetch&add()` primitive

Algorithm: Unbounded stack

```
// REG[i] initialized at ⊥ and NEXT inizialized to 1
def push(v):
    i=NEXT.fetch&add(1)
    REG[i]=v

def pop():
    k=NEXT-1
    for i in [k,1] :
        tmp=REG[i].swap(⊥)
        if tmp≠⊥ :
            return tmp
    return ⊥
```

This implementation of the stack is wait free, but the unbounded register `REG` is not realistic.

We have another implementation that uses an array of register `STACK[k]` that can be read or `compare&set()`. Each value in the stack is a pair (`val,seq_num`). Another register `TOP` is used that contains a triple (`id,val,seq_num`) that represent where in the stack the pair has to be put.

Algorithm: Bounded stack

```
// STACK[i] initialized at (⊥,0) and TOP inizialized to (0,⊥,0)
def conclude(i,v,s):
    tmp = STACK[i].val
    STACK[i].compare&set((tmp,s-1),(v,s))

def push(w):
    (i,v,s) = TOP
    conclude(i,v,s)
    if i=k :
        return FULL
    newtop = (i+1,w,STACK[i+1].seq_num+1)
    if TOP.compare&set((i,v,s),newtop) :
        return OK

def pop():
    while true :
        (i,v,s) = TOP
        conclude(i,v,s)
        if i=0 :
            return EMPTY
        newtop = (i-1,STACK[i-1].val,STACK[i-1].seq_num)
        if TOP.compare&set((i,v,s),newtop) :
            return v
```

The fact that an operation is concluded by the next process ensure correctness even with crashes.

Liveness of stack

The implementation of the stack is non-blocking.

Proof:

Let consider the invocation performed by a process p, if it not terminates it implies that the TOP has been changed, but the only time the TOP is changed is when another operation terminates.

2.3 Obstruction freedom to wait freedom

Like in mutex, we can enhance the liveness properties of a mutex free concurrent algorithm. We define a contention manager, an object that provides 2 operations:

- `need_help(i)`: invoked by a process p_i when it discovers that there is contention
- `stop_help(i)`: invoked by p_i when it terminates the current invocation

In mutex free concurrency we allow process to fail, so we need to take in account that failing without invoking `stop_help(i)` shouldn't be a problem.

2.3.1 From obstuction freedom to non-blocking

To distinguish a long delay from a fail we use a **failure detector** Ω_X. Given a set of processes id X, the failure detector provide a local variable `ev_leader(X)` such that:

1. Validity: it always contain a process id.
2. Eventual leadership: eventually all the local `ev_leader(X)` of non failed processes will contain the same process id, that is not crashed.

The two operation are implemented as:

Algorithm: Non-blocking contention manager

```
// HELP[i] initialized at false
def need_help(i):
    HELP[i]=true
    do:
        X = {j|HELP[j]}
    while ev_leader(X)≠i

def stop_help(i):
    HELP[i]=false
```

Obstuction free to non-blocking

The contention manager transfroms an obstruction free algorithm to a non-blocking one.

Proof:

By contraddiction exists t such that many operations are invoked that never terminate. Let Q be the set of processes that invoked this operations, eventually every process $p_i \in Q$ will set `HELP[i]=t` and will never be modified again (the operation doesn't terminate so it doesn' perform `stop_help(i)`). So, there is a time $t' > t$ for which all processes in Q compute the same set X.

By the definition of Ω_X eventually all process will have the same `ev_leader(X)`, that is the only process allowed to proceed and terminate (obstruction freedom).

Is proved that no wait free implementation of Ω_X exists in an asynchronous system and any number of crashes. To make it possible we need to add constraint:

1. Exists a time t, a time interval Δ and a process p_L such that after t two consecutive writes to a specific register by p_L are at most Δ time apart.

2. Let t be an upper bound on the number of possible crashes. With f real crashes there are at least $t - f$ processes different from p_L with a timer that behaves correctly after t_2. Means that if the timer is set to δ after t_2 it expires at least after δ. This is done because if the processes check to quickly they will wrongly think that p_L is crashed.

The implementation uses an array `PROGRESS` to signal that processes are alive, so every process p_i increases `PROGRESS[i]`. A process p_i suspect p_j if it doesn't see progress after a time interval guessed. The leader becomes the least suspected process.
To guess the correct time interval we use a matrix `SUSPECT` such that `SUSPECT[i,j]` is the number of times that p_i suspected p_j. For all process p_k take the sum of the $t + 1$ minimum values in `SUSPECT[:,k]` and sum them. The time interval guessed is the minimum sum for a p_k.

2.3.2 From obstruction freedom to wait freedom

If we define another failure detector $\diamond P$ that provides each process p_i with a local variable `suspected(i)` such that:

1. Eventual completeness: eventually `supected(i)` contains all the id of crashed processes.

2. Eventual accuracy: eventually `suspected(i)` contains only id of crashed processes.

Failure detectors hierarchy

A failure detector FD1 is stronger than another FD2 if there exists an algorithm that builds FD2 from instances of FD1.

This means that $\diamond P$ is strictly stronger than Ω_X.
Proof:
For every process p_i:

- If $i \notin X$ then `ev_leader(X)` is any id.

- If $i \in X$ then `ev_leader(X)` $= \min_{id}((\Pi -$ `suspected(i)`$) \cap X)$ where Π is the set of all process id.

This proves $\diamond P \geq \Omega_X$. Is not possible that Ω_X is stronger than $\diamond P$ because if it was than consensus will be possible in an asynchronous system.
To implement a contention manager for wait free algorithm we assume a weak timestamp generator, which returns a positive value t and only a finite number of processes can have a timestamp smaller or equal to t.

Algorithm: Wait free contention manager

```
// TS[i] initialized at 0
def need_help(i):
    TS[i]=get_ts()
    do:
        competing = {j|TS[j] ≠ 0 ∧ j ∉ suspected(i)}
        (t,j) = min({(TS[x],x)|x ∈ competing})
    while j≠i

def stop_help(i):
    TS[i]=0
```

Obstuction free to wait free

The contention manager transfroms an obstruction free algorithm to a wait free one.

Proof:
By contraddiction exists an invocation of p_i that never terminates, let t_i be the timestamp and choose the minimum (t_i, i). The set of invocations smaller I is finite (weak ts). For every invocation from a process p_j that crashes, then p_i will forever suspect p_j and will not be in `competing(i)`. So, eventually I will get emptied and for every non crashed process p_k:

- $i \notin$ `suspected(k)`
- $(t_i, i) = \min(\{(\texttt{TS[x]},\texttt{x})|x \in \texttt{competing}\})$

And the invocation of p_i will be executed alone and terminate (obstruction freedom).
To implement $\diamond P$ we need another two constraint:

1. Every non crashed process has eventually an upper bound on write delay.
2. By setting timers, the crashed processes are distinguished from non crashed one.

2.4 Consensus object

Type of an object

The type of an object is:

1. The set of all possible values for states of objects of that type.
2. A set of operation for manipulating the object, each with a specification, description of the conditions under which the operation can be invoked and the effect of the invocation.

An object is universal if every other object can be wait-free implemented by using only object of that type and atomic R/W registers.
The universal object is the consensus object, a one-shot object (any process can access at most once) with only one operation `propose(v)` such that:

1. Validity: the returned value is one of the values proposed by the processes (participants).
2. Integrity: every process decides at most once.
3. Agreement: the decided value is the same for all processes.
4. Wait-freedom: every invocation teminates.

A consensus object by a register X is implemented as:

Algorithm: Consensus

```
// X initialized at ⊥
def propose(v):
    if X=⊥ :
        X=v
    return X
```

The universality of consensus works as follows:

- Given an object O of type Z, each participant runs a local copy of O.
- Create a total order of operation on O.
- All processes follows the order to simulate O locally.

The wait-free construction for determininstc operations uses:

- An unbounded array of consensus objects `CONS`.
- An array `LAST_OP[n]` containing pairs $(\perp, 0)$.
- A local array `last_sn_i[n]` containing the last operation by p_j executed by p_i. This array represent what the process p_i knows of the operations done by other processes.

Algorithm: Wait-free consensus object

```
// executed by p_i
def Z.op(args):
    result_i=⊥
    LAST_OP[i]=(op(args),last_sn_i[i]+1)
    wait result_i≠ ⊥
    return result_i

def Z.simulate():
    k=0
    zi=Z.init()
    while true :
        invoc_i = ϵ
        for ∀j :
            if LAST_OP[j][2]>last_sn_i[j] :
                invoc_i.append((LAST_OP[j][1],j))
        if invoc_i≠ ϵ :
            k++
            exec_i=CONS[k].propose(invoc_i)
            for r in [1,|exec_i|] :
                (zi,res)=δ(zi,exec_i[r][1]) // executes the operation
                j=exec_i[r][2]
                last_sn_i[j]++
                if i=j :
                    result_i=res
```

Correctness of consesus object

To prove that the implementation works we need to prove 3 things:

1. The algorithm is wait free.
2. All operation invoked are executed exactly once.
3. The local copies of z_i behave the same as Z.

Proof 1:

To make a process finish, its result must change, so a consesus object must return a list that contains `(op(args),i)`. The process increases `LAST_OP[i]`, so all lists proposed will contain that element forever. So, eventually the process will execute its operation and change its result.

Proof 2:
After p_i has inserted its operation in `LAST_OP[i]` it stops until that operation is executed, so it cannot invoke another operation. Let `CONS[k]` be the consensus object that contains `(op(args),i)`, every other p_j that partecipate in the k-th consensus increase the value in `last_sn_j[i]` and in the next cycle they will not propose the operation again. So, the operation can only be executed once.
Proof 3:
Each consensus object returns the same list to all processes and the processes use `CONS` in the same order. Given that the operations are determininstic each local copy of Z will behave the same.
In the case of non-determininstic operations we can use 3 solutions:

1. For every pair `(s,op(args))` fix a priori a result.
2. Use additional consensus objects, one for every element of the list.
3. Make the consensus object not only choose the list of invocation but also the state after every the invocation.

2.4.1 Binary and Multivalued consensus

The binary consensus means that there are only two possible proposals, 0 or 1. Multivalues consensus means that the proposals can be arbitrary complex.
A multivalued consensus object can be implemented using binary consensus object as follows:

- We have n processes and n binary consensus rounds.
- At round k, all processes propose 1 if p_k proposed something and 0 otherwise.
- If the consensus object decides 1, all process executes p_k proposal.

Algorithm: Multivalued consensus object (unbounded)

```
// PROP initialized at ⊥
def mv_propose(i,v)):
    PROP[i]=v
    for k in [1,n] :
        if PROP[k]≠ ⊥ :
            res=BC[k].propose(1)
        else :
            res=BC[k].propose(0)
        if res=1 :
            return PROP[k]
    wait forever
```

We can prove that the algorithm works in a simple way:
Let p_x be the first process that writes a proposal, every p_i that participates write its proposal and finds `PROP[x]`$\neq \perp$, so `BC[x]` only recieves proposals equal to 1 and returns 1. So, each process recieves 1 and terminates after at most x cycles.

Let k be the number of possible proposals and $h = \lceil \log k \rceil$ the number of bits needed to binery represent them. There exists an algorithm that decides bit by bit the final outcome creating a multivalued consensus object with bounded values.

Algorithm: Multivalued consensus object (bounded)

```
// PROP initialized at ⊥
def bmv_propose(i,v):
    PROP[i]=binary_repr(v)
    for k in [1,h] :
        P = {PROP[j][k]|PROP[j] ≠ ⊥ ∧ PROP[j][1:k-1]=res[1:k-1]}
        b=P.choose() // a random element of P
        res[k]=BC[k].propose(b)
    return value(res)
```

We only need to prove validity because the other 3 properties (wait freedom, integrity and agreement) are trivial:

By construction of P, for a cycle k, it contains only proposals for which the first $k-1$ bits are equal to the bits decided so far. For whatever $b \in P$ is selected there exists a j that proposed a value that has the k bits equal to the one decided. When $k = h$ the algorithm is concluded.

2.5 Consensus number

Consensus number

The consensus number of an object of type T is the maximum number of process n for which is possible to wait free implement a consensus object using objects of type T.

We define **schedule** the sequence of operation invocation issued by processes and **configuration** the global state of the system at a given execution time. We denote $S(C)$ the configuration obtained starting from configuration C and applying schedule S.

A configuration C is called:

- v-valent: if $S(C)$ decides v for every S.
- monovalent: if exists $v \in \{0, 1\}$ such that C is v-valent.
- bivalent: otherwise.

Fundamental theorem of configuration

Any object that implements binary consensus for n processes must have a bivalent initial configuration.

Proof:

Let C_i be the initial configuration where all p_j with $j \leq i$ propose 1 and the others propose 0. So, C_0 is 0-valent and C_n is 1-valent.

If all starting configuration C_i are monovalent, that means that there is a value i for which C_{i-1} is 0-valent and C_i is 1-valent. The two configuration differ only in the value proposed by p_i.

If p_i is sleeping during C_{i-1} the other processes will decide 0, but the same happens in C_i (the only difference is p_i so they are now equal). So, C_i cannot be 1-valent.

2.5.1 Atomic R/W registers

Consensus number with atomic R/W registers

There exists no way to wait free implement binary consensus with only atomic R/W registers.

Proof:
We have an initial configuration C (bivalent) and let C' be the last bivalent configuration, the one such that $p(C')$ is 0-valent and $q(C')$ is 1-valent (or viceversa).
We have to consider all the possible combination of operation that p and q can issue and prove that we cannot have the 2 monovalent configuration:

1. Operation on different registers:

   We have that $p(q(C')) = q(p(C'))$ but $p(q(C'))$ is 1-valent and $q(p(C'))$ is 0-valent.

2. Both read the same register:

   Works the same as 1.

3. A process p reads and q writes on the same register:

   Only p can distinguish between C' and $p(C')$ so q will behave in the same way both cases, so q will decide 1 even in the configuration $q(p(C'))$. If p was sleeping when it reactivates it decide 1, so $p(C')$ is not 0-valent.

4. Both processes write the same register:

   We have that $p(q(C')) = p(C')$ because the value written by q is lost when p writes after. So, the situation is the same as point 3.

2.5.2 Consensus number of 2 with primitives

With the primitive Test&Set we can implement a consensus object for 2 processes:

Algorithm: Consensus with Test&Set

```
def propose(i,v):
    PROP[i]=v
    if TS.Test&Set()=0 :
        return PROP[i]
    else :
        return PROP[1-i]
```

The algorithm works similarly with the primitives Swap and Fetch&Add.

Consensus number with Test&Set

There exists no way to wait free implement binary consensus for 3 processes with Test&Set registers.

Proof:
Like before let consider C' such that $p(C')$ is 0-valent, $q(C')$ is 1-valent and $r(C')$ is monovalent (0 or 1 is equal).
We have to consider all the possible combination of operation that p,q and r can issue and prove that we cannot have the 3 monovalent configuration:

1. The operation are R/W:

   Works the same as the proof for R/W registers.

2. A process p reads and q test&set on the same register:

   It works the same because only p can distinguish between C' and $p(C')$.

3. Both p and q test&set the same object:

   We have that $p(C')$ is 0-valent and $q(C')$ is 1-valent but the result from the r point of view is that $p(q(C')) = q(p(C'))$ so it will behave in the same way both cases violating the monovalence of one case ($p(C')$ or $q(C')$).

2.5.3 Consensus number of ∞ with Compare&Swap

A consensus object for an arbitrary number of process can be implemented using the Compare&Swap primitive:

Algorithm: Consensus with Compare&Swap

```
def propose(v):
    tmp=CS.compare&swap(⊥,v)
    if tmp=⊥ :
        return v
    else :
        return tmp
```

3

Bisimilarity

3.1 Labeled Transition System

Non deterministic Automaton

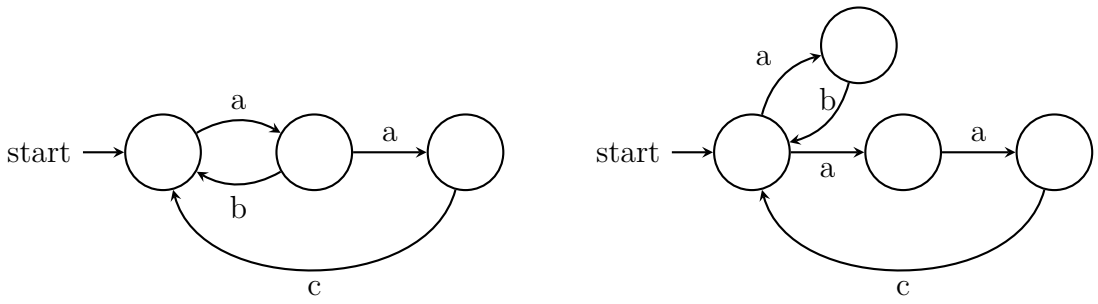
A finite non deterministic automaton is quintuple $M = (Q, A, q_0, F, T)$, where:

- Q i the set of states
- A is the set of actions
- q_0 is the starting state
- F is the set of final state
- T is the transition relation ($T \subseteq Q \times A \times Q$)

The language of an automata $L(M)$ is the set of all the inputs that take M from the starting state q_0 to a final state. Two autoaton are equal if their languages are equal.

Example:

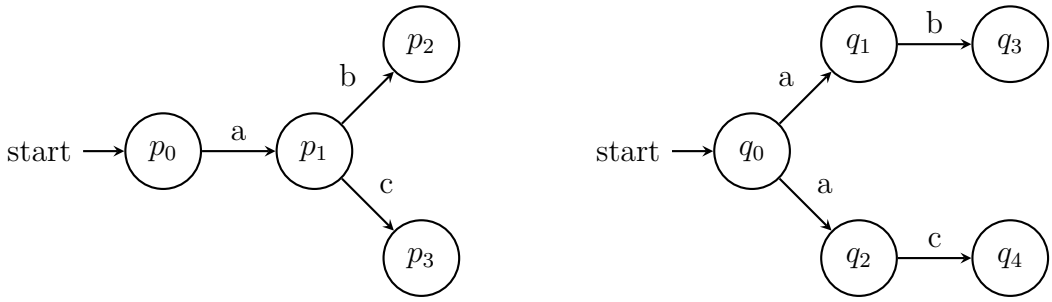
Given this two automaton:



This two automaton has the same language:

$$(a(b + ac))* = (ab + aac)*$$

From an outside observer they behave in this way:



The difference is when the decision to branch is taken and the language equivalence gets rid of that. For concurrent systems this is important, so we need to use something else.

3.1.1 Definition of LTS

Labeled Transition System (LTS) have some main differences from automata:

1. The states of an LTS can be infinite.
2. In an LTS each state can be initial (this corresponds to different possible behaviors of the process).
3. The notion of final states is not informative in LTS.

Labeled Transition System

Given a set of action N an LTS is a pair (Q, T), where:

- Q is the setof states
- T is the transition relation ($T \subseteq Q \times N \times Q$)

We write $s \xrightarrow{a} s'$ instead of $(s, a, s') \in T$.

Two states of LTS will be equivalent if they can perform the same action that lead them in states where this property still holds.

Bisimilarity

Given an LTS (Q, T), a binary relation $S \subseteq Q \times Q$ is a **simulation** if and only if:

$$\forall (p, q) \in S \; \forall p \xrightarrow{a} p' \; \exists q \xrightarrow{a} q' | (p', q') \in S$$

If $(p, q) \in S$ we say that p is simulated by q.
The relation S is a **bisimulation** if also S^{-1} is a simulation, where:

$$S^{-1} = \{(q, p) | (p, q) \in S\}$$

If exists a bisimulation S we say that p and q are **bisimilar** (written $p \sim q$).

Example:
Given this two LTS:

To prove that $p_0 \sim q_0$ we need to check that the following relations are simulations:

$$S = \{(p_0, q_0), (p_1, q_1), (p_2, q_2), (p_0, q_2)\}$$
$$S^{-1} = \{(q_0, p_0), (q_1, p_1), (q_1, p_2), (q_2, p_0)\}$$

Bisimilarity as equivalence

Bisimilarity is an equivalence relation

Proof:

1. Reflexivity:

   The relation $S = (p, p)|p \in Q$ is a bisimulation, because S is a simulation and also $S^{-1} = S$ is a simulation.

2. Symmetry:

   For every pair $(p, q) \in S$, by definition of bisimulation also S^{-1} is a simulation and $(q, p) \in S^{-1}$, so $q \sim p$.

3. Transitivity:

   Consider the following relation:

   $$S = \{(x, z)|\exists y|(x, y) \in S_1 \wedge (y, z) \in S_2\}$$

   where S_1 and S_2 are bisimulations.

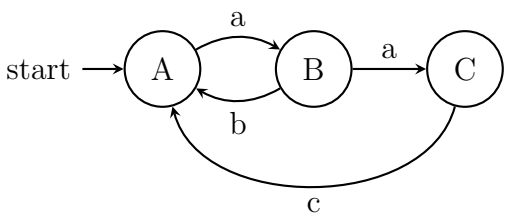
   For every $x \xrightarrow{a} x'$ there exists an $y \xrightarrow{a} y'$ such that $(x', y') \in S_1$ and also $z \xrightarrow{a} z'$ such that $(y', z') \in S_2$. So, from $x \xrightarrow{a} x'$ we found $z \xrightarrow{a} z'$ such that $(x', z') \in S$.

3.1.2 Syntax for non deterministic processes

If the number of states grows, we need to find another way to write down LTS in a more readable way. We can describe an LTS by a system of equations and obtain a recursive definition of the machine behaviour.

Example:

Given the LTS:



We can write the behaviour by a system of equations:

$$\begin{cases} A = a.B \\ B = b.A + a.C \\ C = c.A \end{cases}$$

where . is the sequantial composition and + the non deterministic choice. We can obtain the recursive definition of the machine behaviour:

$$A(a, b, c) = a.(b.A(a, b, c) + a.c.A(a, b, c))$$

This machine definition is parametric in a, b, c.

Set of non deterministic processes

The set of non deterministic processes is given by the following grammar:

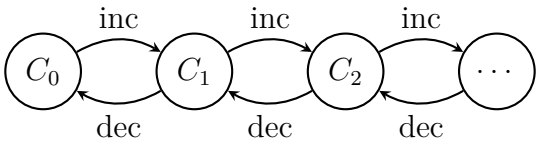
$$P = \sum_{i \in I} \alpha_i.P_i | A(a_1, \ldots, a_n)$$

where I is the finite set of indices and $\alpha_i \in \mathcal{A}$, for every $i \in I$.

Example:
Take as example a counter, starting from 0. With the actions *inc* and *dec*, this can be modeled:

$$\begin{cases} C_0 = inc.C_1 \\ C_i = inc.C_{i+1} + dec.C_{i-1} \quad i > 0 \end{cases}$$

The resulting LTS is:



3.2 CCS

To model a concurrent system correctly we need to model two main features:

1. Simultaneous execution of different processes
2. Interaction between processes

Given a set of events, we denote:

- a the consumption of event a
- $\overline{a}$ the production of event a

The actions a and $\overline{a}$ are complementary and let two processes synchronize on event a. The synchronization is not observable from outside, but produce a silent event τ.
The set of actions that we consider is $\mathcal{A} = \mathcal{N} \cup \overline{\mathcal{N}} \cup \tau$.

Set of CCS processes

The set of CCS processes is defined by the following grammar:

$$P = \left(\sum_{i \in I} \alpha_i.P_i | A(a_1, \ldots, a_n) \right) P|Q \ |P \backslash a$$

where I is the finite set of indices and $\alpha_i \in \mathcal{A}$, for every $i \in I$. $P|Q$ means the parallel execution of P and Q and $P\backslash a$ means that a is visible only from within P.

Inference rules

There are two equal definitions of the inference rules:

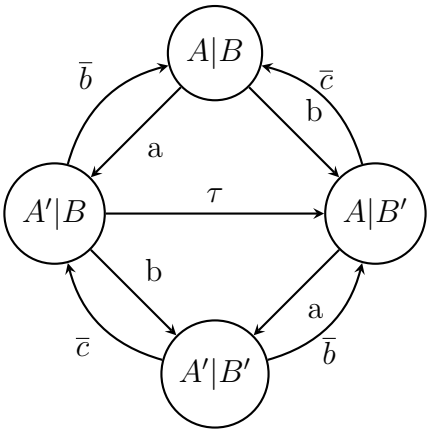
1. $$\sum_{i \in I} \alpha_i.P_i \xrightarrow{\alpha_j} P_j \quad \forall i \in I$$

2. $$\frac{P\{a_1/x_n, \ldots, a_n/x_n\} \xrightarrow{\alpha} P'}{A(a_1, \ldots, a_n) \xrightarrow{\alpha} P'} A(x_1, \ldots, x_n) \triangleq P$$

If two processes work in parallel they must synchronize, so one must do a and the other $\overline{a}$ for some $a \in \mathcal{A}$.

Example:

Given this:



The processes definitions are:

$$\begin{cases} A \triangleq a.A' \\ A' \triangleq \overline{b}.A \\ B \triangleq b.B' \\ B' \triangleq \overline{c}.B \end{cases}$$

There are 3 possibilities:

1. Only A does something:
$$\frac{\frac{a.A' \xrightarrow{a} A'}{A \xrightarrow{a} A'} A \triangleq a.A'}{A|B \xrightarrow{a} A'|B}$$

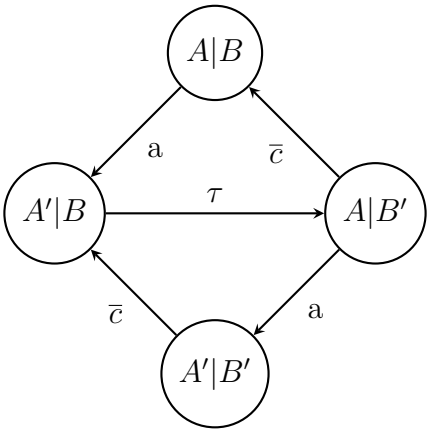
2. Only B does something:
$$\frac{\frac{b.B' \xrightarrow{b} B'}{B \xrightarrow{b} B'} B \triangleq b.B'}{A|B \xrightarrow{b} A|B'}$$

3. They both work in parallel and synchronize ($\overline{a}$ and c don't exist, so they must synchronize with b and $\overline{b}$):
$$\frac{\frac{\overline{b}.A \xrightarrow{\overline{b}} A}{A' \xrightarrow{\overline{b}} A} A' \triangleq \overline{b}.A \quad \frac{\frac{b.B' \xrightarrow{b} B'}{B \xrightarrow{b} B'} B \triangleq b.B'}{}}{A'|B \xrightarrow{\tau} A|B'}$$

In the construction of the LTS we loose the consciousness of the parallel.
The usefulness of the parallel is two-fold:

- it is the fundamental operator in concurrency theory
- it allows for a compact and intuitive writing of processes

Example:
Given the previous example, we restrict b and so we remove the transitions involving b or $\overline{b}$. The transition τ remains because it signal that the synchronization has happened even if the events that cause the synchronization are hide. The LTS is changed, becoming:



Now we can pass from $A|B$ to $A'|B$ performing a but we can't pass from $A|B$ to $A|B'$ performing b.

3.2.1 Semaphor

An n-ary semaphor (denoted as $S^{(n)}(p, v)$) is a process used to ensure that at most n istances of the same activity are in execution. An activity is started by an action p (down) and terminated by an action v (up).

The definition of a unary semaphor is:

$$\begin{cases} S^{(1)} \triangleq p.S_1^{(1)} \\ S_1^{(1)} \triangleq v.S^{(1)} \end{cases}$$

The definition of a binary semaphor is:

$$\begin{cases} S^{(2)} \triangleq p.S_1^{(2)} \\ S_1^{(2)} \triangleq p.S_2^{(2)} + v.S^{(2)} \\ S_2^{(2)} \triangleq v.S_1^{(2)} \end{cases}$$

We can show that there is a bisimulation between $S^{(2)}$ and the implementation with two processes $S^{(1)}$:

$$S^{(1)}|S^{(1)} \sim S^{(2)}$$

To show the equivalence, we can show that this relation is a bisimulation:

$$R = \{(S^{(1)}|S^{(1)}, S^{(2)}), (S_1^{(1)}|S^{(1)}, S_1^{(2)}), (S^{(1)}|S_1^{(1)}, S_1^{(2)}), (S_1^{(1)}|S_1^{(1)}, S_2^{(2)})\}$$

But there is no synchronization between the two $S^{(1)}$ so they cannot work in parallel.

3.2.2 Image Finiteness

Image Finiteness

For a state of a process P, the number of outgoing arrows is finite:

$$\left|\{P' | \exists \alpha \; P \xrightarrow{\alpha} P'\}\right| < \infty$$

Proof:

For every process P, let $\mathcal{T}(P)$ be the set of derivation trees generated from any transition $P \xrightarrow{\alpha} P'$. We denote k_P the maximum height of a tree in $\mathcal{T}(P)$. We do the proof by induction of k_P:

- Base case:

  In this case we simply have that:

  $$\left|\{P' | \exists \alpha \; P \xrightarrow{\alpha} P'\}\right| = |\{P_i | i \in I\}| \leq |I| < \infty$$

- Inductive step:

  We need to consider 3 cases:

  1. $P \triangleq Q \backslash a$: all inferences for a reduction from P must have as last rule the restriction one:

     $$\frac{Q \xrightarrow{\alpha} Q'}{Q \backslash a \xrightarrow{\alpha} Q' \backslash a} \alpha \notin \{a, \overline{a}\}$$

     Since $k_Q < k_P$ we know that:

     $$\left|\{Q' | \exists \alpha \; Q \xrightarrow{\alpha} Q'\}\right| < \infty$$

And we have that:

$$\{P' | \exists \alpha\ P \xrightarrow{\alpha} P'\} \subseteq \{Q' \backslash a | \exists \alpha\ Q \xrightarrow{\alpha} Q'\}$$

The right set is finite, so also the left one is finite.

2. $P \triangleq A(a_1, \ldots, a_n)$: all inferences of P will be in the form:

$$\frac{Q\{a_1/x_1, \ldots, a_n/x_n\} \xrightarrow{\alpha} Q'}{A(a_1, \ldots, a_n) \xrightarrow{\alpha} Q'} A(x_1, \ldots, x_n) \triangleq Q$$

Since $k_Q < k_P$ we know that:

$$\left|\{R | \exists \alpha\ Q\{a_1/x_1, \ldots, a_n/x_n\} \xrightarrow{\alpha} R\}\right| < \infty$$

And we have that:

$$\{P' | \exists \alpha\ P \xrightarrow{\alpha} P'\} = \{R | \exists \alpha\ Q\{a_1/x_1, \ldots, a_n/x_n\} \xrightarrow{\alpha} R\}$$

The right set is finite, so also the left one is finite.

3. $P \triangleq P_1 | P_2$: we have 4 possible inference trees, one where P_1 continue, one where P_2 continue and the last two where they synchronize:

$$\begin{aligned}\{P' | \exists \alpha\ P \xrightarrow{\alpha} P'\} \quad = \quad & \{P_1' | P_2\ | \exists \alpha\ P_1 \xrightarrow{\alpha} P_1'\} \cup \\ & \{P_1 | P_2'\ | \exists \alpha\ P_2 \xrightarrow{\alpha} P_2'\} \cup \\ & \{P_1' | P_2'\ | \exists \alpha\ P_1 \xrightarrow{\alpha} P_1' \wedge P_2 \xrightarrow{\overline{\alpha}} P_2'\} \cup \\ & \{P_1' | P_2'\ | \exists \alpha\ P_1 \xrightarrow{\overline{\alpha}} P_1' \wedge P_2 \xrightarrow{\alpha} P_2'\}\end{aligned}$$

All the sets are finite, so also the left one is finite.

3.2.3 Properties of bisimilarity

Let us consider a renaming function $\sigma : \mathcal{N} \to \mathcal{N}$ and we define the result of applying σ:

1. $\sigma(\overline{a}) = \overline{\sigma(a)}$
2. $\sigma(\sum_{i \in I} \alpha_i.P_i) = \sum_{i \in I} \sigma(\alpha_i).\sigma(P_i)$
3. $\sigma(P \backslash a) = \sigma(P) \backslash \sigma(a)$
4. $\sigma(P_1 | P_2) = \sigma(P_1) | \sigma(P_2)$
5. $\sigma(A(a_1, \ldots, a_n)) = A^\sigma(\sigma(a_1), \ldots, \sigma(a_n))$

And for every definition $A(x_1, \ldots, x_n) \triangleq P$ we assume the definition:

$$A^\sigma(\sigma(x_1), \ldots, \sigma(x_n)) \triangleq \sigma(P)$$

Injective renaming working

For every injective renaming $\sigma : \mathcal{N} \to \mathcal{N}$, if $P \xrightarrow{\alpha} P'$ then $\sigma(P) \xrightarrow{\sigma(\alpha)} \sigma(P')$

Proof:
The proof is done by induction on the height of the inference $P \xrightarrow{\alpha} P'$.

- Base case:

  The base case is $P = \sum_{i \in I} \alpha_i.P_i \xrightarrow{\alpha_j} P_j$. We have that $\sigma(P) = \sum_{i \in I} \sigma(\alpha_i).\sigma(P_i)$ and then $\sigma(P) \xrightarrow{\sigma(\alpha_j)} \sigma(P_j)$ for every $j \in I$.

- Inductive step:

  We have to consider 3 cases:

  1. $P \triangleq A(a_1, \ldots, a_n), Q \triangleq A(x_1, \ldots, x_n)$ and $Q\{a_1/x_1, \ldots, a_n/x_n\} \xrightarrow{\alpha} P'$: by definition we have that $\sigma(P) = A^{\sigma}(\sigma(a_1), \ldots, \sigma(a_n))$ and by inductive hypothesis we have that:

  $$\sigma(Q)\{\sigma(a_1)/\sigma(x_1), \ldots, \sigma(a_n)/\sigma(x_n)\} \xrightarrow{\sigma(\alpha)} \sigma(P')$$

  2. $P \triangleq Q\backslash a, Q \xrightarrow{\alpha} Q'$ and $\alpha \notin \{a, \overline{a}\}$: by definition we have $\sigma(Q\backslash a) = \sigma(Q)\backslash\sigma(a)$ and since $\sigma(\alpha) \notin \{\sigma(a), \overline{\sigma(a)}\}$ we have that $\sigma(P) \xrightarrow{\sigma(\alpha)} \sigma(P')$.
  3. $P \triangleq P_1|P_2$: we have that $\sigma(P) \triangleq \sigma(P_1)|\sigma(P_2)$ so it works the same as before.

We can see that a process $a.P\backslash a \sim 0$ because they both don't have outgoing arrows. The same works for $\overline{a}.P\backslash a \sim 0$.

Idempotency of sum

Given a process P, we have that:

$$a.P + a.P + M \sim a.P + M$$

where M is a sum $\sum_{i \in I} \beta_i.P_i$.

Proof:
The two definitions have the same outgoing arrows, so the set:

$$S = \{(a.P + a.P + M, a.P + M)\} \cup Id$$

is a bisimulation.

3.2.4 Congruence

We define an execution context as a process with a hole ($\square$) that can be filled by a process P. This filling is denoted by $C[P]$. Intuitively if two processes P and Q are equivalent they can be interchangeably used in any execution context.

Set of CCS context

The set C of CCS contexts is given by the following grammar:

$$C = \square \mid C|P \mid C\backslash a \mid \alpha.C + M$$

Bisimilarity as congruence

An equivalence relation $\mathcal{R}$ is a congruence if and only if:

$$\forall (P,Q) \in \mathcal{R}, \forall C\ (C[P], C[Q]) \in \mathcal{R}$$

So for bisimilarity, we have that:

$$\forall P \sim Q, \forall C\ C[P] \sim C[Q]$$

Proof:
To prove this we need to prove all the possibilities for a context:

- If $P \sim Q$, then $\alpha.P + M \sim \alpha.Q + M$ for all α and M:

  We need to show that:

  $$S = \left\{ (\alpha.P + M, \alpha.Q + M) \middle| P \sim Q, \alpha \in \mathcal{A}, M = \sum_{i \in I} \alpha_i.P_i \right\} \cup \sim$$

  is a simulation. We have two cases:

  1. $\alpha.P + M \xrightarrow{\alpha} P$: also $\alpha.Q + M$ with action α will become Q that is bisimilar to P.
  2. $\alpha.P + M \xrightarrow{\alpha_j} P_j$: also $\alpha.Q + M$ can reply with the same action and by reducing to the same process P_j.

- If $P \sim Q$, then $P\backslash a \sim Q\backslash a$ for all a:

  We need to show that:

  $$S = \{(P\backslash a, Q\backslash a) | \forall P \sim Q, \forall a \in \mathcal{N}\}$$

  is a simulation. We have that $P\backslash a \xrightarrow{\alpha} P'$ where $\alpha \notin \{a, \overline{a}\}$, so Q can also perform the same action α and reduce to the process Q'. Since $P \sim Q$ we have that $P' \sim Q'$.

- If $P \sim Q$, then $P|R \sim Q|R$ for all R:

  We need to show that:

  $$S = \{(P|R, Q|R) | P \sim Q\}$$

  is a simulation. We have 3 cases:

  1. Only P evolves, so $P|R \xrightarrow{\alpha} P'|R$: since $P \sim Q$ we have that $Q \xrightarrow{\alpha} Q'$ and $P' \sim Q'$, so also $P'|R \sim Q'|R$.
  2. Only R evolves, so $P|R \xrightarrow{\alpha} P|R'$: trivially $Q|R \xrightarrow{\alpha} Q|R'$ and $P|R' \sim Q|R'$.
  3. They both evolve and synchronize, so $P \xrightarrow{a} P', R \xrightarrow{\overline{a}} R'$ (or viceversa) and $P|R \xrightarrow{\tau} P'|R'$: since $P \sim Q$ we have that $Q \xrightarrow{a} Q'$ and $P' \sim Q'$, so $Q|R \xrightarrow{\tau} Q'|R'$ and $P'|R' \sim Q'|R'$.

After proving that the bisimilarity is propagated by the context we can prove the starting lemma by induction on the structure of C:

- Base case:

  $C = \square$ and $C[P] = P\ \forall P$, so trivially $C[P] \sim C[Q]$.

- Inductive step:

  Using the previous lemmata every structure of C contains another context C' that is smaller than C and so $C'[P] \sim C'[Q]$.

3.3 Weak bisimilarity

We define a less stringent definition of bisimilarity, that does not take account for non observable internal actions.
The idea of this new ewuivalence is:

- A visible action must be replied with the same action and some internal action.
- An internal action must be replied with 0 or more internal actions.

We define that $P \implies P'$ if and only if there exist $P_0, \ldots, P_k$ such that $P \xrightarrow{\tau} P_0 \xrightarrow{\tau} \ldots \xrightarrow{\tau} P_k \xrightarrow{\tau} P'$.
We define the relation $\overset{\hat{\alpha}}{\Longrightarrow}$ such that:

- If $\alpha = \tau$ then $\overset{\hat{\alpha}}{\Longrightarrow} = \implies$.
- Otherwise $\overset{\hat{\alpha}}{\Longrightarrow} \triangleq \implies \xrightarrow{\alpha} \implies$. This means soe τ followed by α and followed by some τ again.

Weak bisimilarity

S is a weak simulation if and only if:

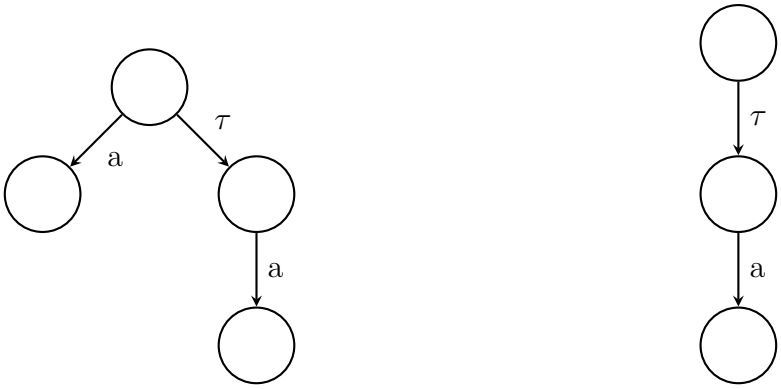
$$\forall (p,q) \in S \; \forall p \xrightarrow{p'} \; \exists q \overset{\hat{\alpha}}{\Longrightarrow} q' | (p',q') \in S$$

The relation S is a weak bisimulation if also S^{-1} is a weak simulation.
If exists a bisimulation S we say that p and q are weakly bisimilar (written $p \approx q$).

We can say that $\approx$:

1. Is an equivalence.
2. Is a weak bisimulation.
3. Is a congruence.
4. $\sim \subset \approx$.

Example:
The following LTS are weak bisimilar:



Types of weak bisimulation

Given a process P and any sum M and N:

1. $P \approx \tau.P$
2. $M + N + \tau.N \approx M + \tau.N$
3. $M + \alpha.P + \alpha(N + \tau.P) \approx M + \alpha(N + \tau.P)$

Proof:
Is easy to show that exists a weak simulation for each of the 3 types:

1. $S = \{(P, \tau.P)\} \cup Id$
2. $S = \{(M + N + \tau.N, M + \tau.N)\} \cup Id$
3. $S = \{(M + \alpha.P + \alpha(N + \tau.P), M + \alpha(N + \tau.P))\} \cup Id$

Example:
A factory can handle 3 kinds of works: easy (E), medium (M) and difficult (D). An activity consist in recieving an input and producing an output, so the specification are the following:

$$\begin{cases} A \triangleq i_E.A' + i_M.A' + i_D.A' \\ A' \triangleq \overline{o}.A \end{cases}$$

A factory is the parallel composition of two activities:

$$Factory \triangleq A|A$$

A possible implementation is done by using two workers that work in parallel and use machine based on the kind of work:

- Easy work: no machine.
- Medium work: general or special machine.
- Difficult work: special machine.

The specification of a worker is:

$$\begin{cases} W \triangleq i_E.W_E + i_M.W_M + i_D.W_D \\ W_E \triangleq \overline{o}.W \\ W_M \triangleq \overline{rg}.\overline{lg}.W_E + \overline{rs}.\overline{ls}.W_E \\ W_D \triangleq \overline{rs}.\overline{ls}.W_E \end{cases}$$

Where rg, rs are used to request a machine and lg, ls to leave the machine. The specification of the machines are:

$$\begin{cases} S \triangleq rs.S' \\ S' \triangleq ls.S \\ G \triangleq rg.G' \\ G' \triangleq lg.G \end{cases}$$

The system is given by:

$$Workers \triangleq (W|W|G|S)\backslash\{rg, rs, lg, ls\}$$

We can show the following weak bisimilarity:

$$Factory \approx Workers$$

Let $N = \{rg, rs, lg, ls\}$ and $x, y \in \{E, M, D\}$, the weak bisimulation R is composed by this pairs:

1. $(A|A, (W|W|G|S)\backslash N)$
2. $(A|A', (W|W_x|G|S)\backslash N)$
3. $(A|A', (W|\overline{lg}.W_E|G'|S)\backslash N)$
4. $(A|A', (W|\overline{ls}.W_E|G|S')\backslash N)$
5. $(A'|A', (W_y|W_x|G|S)\backslash N)$
6. $(A'|A', (W_y|\overline{lg}.W_E|G'|S)\backslash N)$
7. $(A'|A', (W_y|\overline{ls}.W_E|G|S')\backslash N)$
8. $(A'|A', (\overline{lg}.W_E|\overline{ls}.W_E|G'|S')\backslash N)$

This relation contains 3 pairs for the 2nd, 6th and 7th form (one for every x or y) and 9 pair for the 5th form (one for each pair x, y), for a total of 34 pairs.

3.4 Inference system

To check bisimilarity for processes without recursion we can use an inference system made up of axioms and inferenxe rules. This inference system has 2 properties:

- Soundness: whatever I infer is bisimilar
- Completeness: whatever is bisimilar, can be infered

3.4.1 Strong bisimilarity

The axioms that make up this inference systems for bisimilarity are:

- Axioms for sum:
  1. $\vdash M + 0 = M$
  2. $\vdash M_1 + M_2 = M_2 + M_1$
  3. $\vdash M_1 + (M_2 + M_3) = M_1 + M_2 + M_3$
  4. $\vdash M + M = M$
- Axioms for parallel composition:
  1. $\vdash \sum_i \alpha_i.P_i | \sum_j \beta_j.Q_j = \sum_i \alpha(P_i | \sum_j \beta_j.Q_j) + \sum_j \beta_j(\sum_i \alpha_i.P_i | Q_j) + \sum_{\alpha_i = \overline{\beta_j}} \tau(P_i | Q_j)$
- Axioms for restriction:
  1. $\vdash 0 \backslash a = 0$
  2. $\vdash (\sum_i \alpha_i.P_i) \backslash a = \sum_i (\alpha_i.P_i) \backslash a$
  3. $\vdash (\alpha.P) \backslash a = \begin{cases} 0 & \alpha \in \{a, \overline{a}\} \\ \alpha.(P \backslash a) & \text{otherwise} \end{cases}$

The inference rules are the one to make this system an equivalence and a congruence:

1. Reflexivity: $\vdash P = P$
2. Simmetry: $\frac{\vdash P = Q}{\vdash Q = P}$
3. Transitivity: $\frac{\vdash P = Q \quad \vdash Q = R}{\vdash P = R}$
4. Congruence: $\frac{\vdash P = Q}{\vdash C[P] = C[Q]}$

Soundness

This inference system has the soundness property, so:

$$\vdash P = Q \implies P \sim Q$$

Proof:
For every axiom in the form $\vdash X = Y$, we can consider the relation $\{(X, Y)\} \cup Id$ and prove that is a bisimulation. The system is an equivalence and a congruence so the inference rules hold for bisimilarity.

Standard form of a process

A process P is in standard form if and only if:

$$P \triangleq \sum_i \alpha_i.P_i$$

And each P_i is in standard form.
For every process P exists another process P' in standard form, such that $\vdash P = P'$.
So, every P is representable in standard form.

Proof:
We prove the existence of P' by induction on the structure of P:

- Base case:

  $P \triangleq 0$ is already in standard form.

- Inductive step: We have to consider 3 cases:

  1. $P \triangleq P_1|P_2$

     By induction we have P_1', P_2' in standard form such that $\vdash P_1 = P_1'$ and $\vdash P_2 = P_2'$, where $P_1' = \sum_i \alpha_i.R_i$ and $P_2' = \sum_j \beta_j.Q_j$.

     By transitivity we have that:

     $$\overbrace{P_1|P_2}^{P} = \sum_i \alpha_i.R_i|\sum_j \beta_j.Q_j$$

     $$= \sum_i \alpha(R_i|\sum_j \beta_j.Q_j) + \sum_j \beta_j(\sum_i \alpha_i.R_i|Q_j) + \sum_{\alpha_i=\overline{\beta_j}} \tau(R_i|Q_j) = \cdots = P'$$

     where the elimination of the paallel is repeated until there are no more parallel in the process.

  2. $P \triangleq \sum_i .P_i$

     By induction we have that $\forall P_i \ \exists P_i'$ in standard form such that $\vdash P_i = P_i'$. From $\vdash P_1 = P_1'$ with the context:

     $$\alpha_1.\Box + \sum_{i\in I\setminus\{1\}} \alpha_i.P_i$$

     we have that:

     $$\vdash \alpha_1.P_1 + \sum_{i\in I\setminus\{1\}} \alpha_i.P_i = \alpha_1.P_1' + \sum_{i\in I\setminus\{1\}} \alpha_i.P_i$$

     We can repeat this process for each P_i and we have that:

     $$\underbrace{\sum_{i\in I} \alpha_i.P_i}_{P} = \underbrace{\sum_{i\in I} \alpha_i.P_i'}_{P'}$$

  3. $P \triangleq Q\backslash a$

     By induction we have a Q' in standard form such that $\vdash Q = Q'$. From this and by congruence, we have that:

     $$\overbrace{Q\backslash a}^{P} = Q'\backslash a = \sum_i (\alpha_i.R_i)\backslash a = \sum_{i\in I} \alpha_i(R_i\backslash a)$$

     where $I' = \{i \in I | \alpha_i \notin \{a, \overline{a}\}\}$ and the elimination of restriction is repeated until the operator is removed.

Completeness

This inference system has the soundness property, so:

$$P \sim Q \implies \vdash P = Q$$

Proof:

By the previous definition we have that $\vdash P = P'$ and $\vdash Q = Q'$ where:

$$P' = \sum_i \alpha_i.P_i \quad Q' = \sum_j \beta_j.Q_j$$

So, we have to prove that $\vdash P' = Q'$ and we do that by induction on the maximum height of the syntactic tree that describes P' and Q':

- Base case:

  $P' = Q' = 0$ is trivial.

- Induction:

  For an action α_1 we have that:

  $$P' \xrightarrow{\alpha_1} P_1 \implies P \xrightarrow{\alpha_1} \hat{P} \text{ and } P_1 \sim \hat{P} \implies Q \xrightarrow{\alpha_1} \hat{Q} \text{ and } \hat{P} \sim \hat{Q}$$

  $$\implies Q' \xrightarrow{\alpha_1} Q'' \text{ and } \hat{Q} \sim Q''$$

  By definition of Q' must exists $\beta_{j_1} = \alpha_1$ and $Q'' = Q_{j_1}$ and by induction $P_1 \sim Q_{j_1} \implies \vdash P_1 = Q_{j_1}$.

  We now consider the context:

  $$C = \alpha_1.\Box + \sum_{i \in I \setminus \{1\}} \alpha_i.P_i$$

  And we can say that:

  $$\underbrace{\alpha_1.P_1 + \sum_{i \in I \setminus \{1\}} \alpha_i.P_i}_{P'} = \beta_{j_1}.Q_{j_1} + \sum_{i \in I \setminus \{1\}} \alpha_i.P_i$$

  Doing the same for every P_i we can prove that:

  $$\vdash P' = \sum_i \beta_{j_i}.Q_{j_i}$$

  In the same way we can prove that:

  $$\vdash Q' = \sum_j \alpha_{i_j}.P_{i_j}$$

  And from summing the equalities:

  $$\vdash P' + \sum_j \alpha_{i_j}.P_{i_j} = Q' + \sum_i \beta_{j_i}.P_{j_i}$$

  By idempotency we have $\vdash P' = Q'$ and consequently $\vdash P = Q$.

3.4.2 Weak bisimilarity

We can create an inference system also for weak bisimilarity, in which all the rules of the previous inference systems hold and adding the axioms for *tau*:

1. $\vdash \alpha.P = \alpha.\tau.P$
2. $P + \tau.P = P$
3. $\alpha.(P + \tau.Q) = \alpha.(P + \tau.Q) + \alpha.Q$

Example:
Consider a server that recieves a request for sending a message and delivers a confirm of the reception. The specification of the server is:

$$Spec = send.\overline{rcv}$$

The server can be implemented by 3 process in parallel:

$$\left.\begin{array}{l} S = send.\overline{put} \\ M = put.\overline{go} \\ R = go.\overline{rcv} \end{array}\right\} Impl = (S|M|R)\backslash\{put, go\}$$

To prove that the specification and the implementation are weekly bisimilar we need to apply the axioms and inference rules of the system:

1. Axiom of parallel:
$$\vdash (S|M|R) = send.(\overline{put}|M|R) + put.(S|\overline{go}|R) + go.(S|M|\overline{rcv})$$
2. Axiom 2 of restriction:
$$\vdash Impl = (send.(\overline{put}|M|R) + put.(S|\overline{go}|R) + go.(S|M|\overline{rcv}))\backslash\{put, go\}$$
3. Axiom 3 of restriction:
$$\vdash Impl = send.(\overline{put}|M|R)\backslash\{put, go\} + 0 + 0$$
4. Using a similar way:
$$\vdash Impl = send.\tau.(\overline{go}|R)\backslash\{put, go\}$$
5. Axiom 1 of τ:
$$\vdash Impl = send.(\overline{go}|R)\backslash\{put, go\} = send.\tau.(\overline{rcv})\backslash\{put, go\}$$
6. Axiom 1 of τ:
$$\vdash Impl = send.(\overline{rcv})\backslash\{put, go\} = send.\overline{rcv}$$

So the specification and implementation are weakly bisimilar.

3.5 Logic formulas

To check if two processes are not bisimilar we use formulas and satisfiability to prove that one process can satisfy at least one formula that the other process cannot satisfy.

Logic for strong bisimilarity

The formulas that can be written with this logic are in the form:

$$\varphi = True \mid \neg\varphi \mid \varphi \wedge \varphi \mid \diamond a\varphi$$

A process P satisfies a formula φ written as $P \models \varphi$ if and only if $(P, \varphi) \in \models$.

The relation $\models$ can be inductively defined:

- $P \models True$
- $P \models \neg\varphi \iff P \not\models \varphi$
- $P \models \varphi_1 \wedge \varphi_2 \iff P \models \varphi_1 \wedge P \models \varphi_2$
- $P \models \diamond a\varphi \iff \exists P' | P \xrightarrow{a} P' \wedge P' \models \varphi$

We consider also the possibility to have infinite many conjunction with the operator:

$$\bigwedge_{i \in I} \varphi_i$$

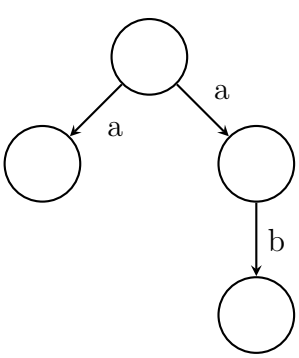
From the operators before we can simply derive also $False$ and logical operator such as $\vee$ and $\Longrightarrow$.
Another important operator is the box (representing the for each) that can define:

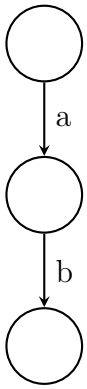
$$\neg \diamond a\varphi = \Box a \neg\varphi$$

The box operator means that:

- $P \models \Box a\varphi \iff \forall P' | P \xrightarrow{a} P' \implies P' \models \varphi$
- $P \models \Box aF \iff \nexists P' | P \xrightarrow{a} P'$, so P cannot perform a

Example:
Given the following processes:





We can see that using the formula:

$$\varphi = \diamond a \Box b F$$

we have that $P_1 \models \varphi$ and $P_2 \not\models \varphi$, so the two processes does not satisfy the same formulas.

Bisimilarity on set of formulas

Given the set of fomulas satisfied by a process defined as:

$$L(P) = \{\varphi | P \models \varphi\}$$

We have that:

$$P \sim Q \iff L(P) = L(Q)$$

Proof:

1. $P \sim Q \implies L(P) = L(Q)$

   We prove this by induction on the syntax tree of the formula, showing that $\varphi \in L(P) \iff \varphi \in L(Q)$:

   - Base case:
     The base case is $\varphi = T$ and it belongs to $L(P)$ and $L(Q)$.
   - Inductive step:
     We assume the thesis for every syntax tree of height h and we consider the possible cases for the $h+1$-th operator:

     1.1 $\varphi = \bigwedge_{i \in I} \varphi_i$:
     Let assume $\varphi \in L(P)$, it means that $\forall i \varphi_i \in L(P)$. Every formula has a syntax tree with height h, so by ipothesis $\forall i \varphi_i \in L(Q)$, so $\varphi \in L(Q)$.

     1.2 $\varphi = \neg \varphi'$:
     In this case $\varphi' \notin L(P)$ and the height of its syntax tree is h, so $\varphi' \notin L(Q)$ and $\varphi \in L(Q)$.

     1.3 $\varphi = \diamond a \varphi'$:
     In this case $\exists P' | P \xrightarrow{a} P'$ and $\varphi' \in L(P')$. If $P \sim Q$ means that $\exists Q' | Q \xrightarrow{a} Q'$ and $P' \sim Q'$. The height of φ' is h so $\varphi' \in L(Q')$ and by definition $Q \models \diamond a \varphi' = \varphi$.

2. $P \sim Q \impliedby L(P) = L(Q)$:

   We can prove that the relation:

   $$R = \{(P, Q) | L(P) = L(Q)\}$$

   is a simulation.

   Let $P \xrightarrow{a} P'$ and consider the formula:

   $$\varphi = \diamond a \varphi' \quad \varphi' = \bigwedge_{\varphi'' \in L(P')} \varphi''$$

   By construction $P \models \varphi$ and also $Q \models \varphi$ because $L(P) = L(Q)$. It means that $\exists Q' | Q \xrightarrow{a} Q'$ and $Q' \models \varphi'$. So, we have that:

   $$\forall \varphi'' \in L(P') \; \varphi'' \in L(Q') \implies L(P') \subseteq L(Q')$$

   The inclusion cannot be proper ($L(P') \subset L(Q')$) because for every formula $\neg\varphi \in L(P')$ its not possible that $\varphi \in L(Q')$ because Q' cannot satisfy both φ and $\neg\varphi$.

3.5.1 Sub-logics

Negation-free logic

The formulas that can be written with this logic are in the form:

$$\varphi = True \mid \varphi \wedge \varphi \mid \ \diamond a\varphi$$

We write $L_{\neg}(P)$ the set of formulas satisfied by P.
We can prove that:

$$P \text{ simulates } Q \iff L_{\neg}(P) = L_{\neg}(Q)$$

This can be done similarly to the previous proof with the difference that without $\neg$ we don't have $L(P') = L(Q')$ but only $L(P') \subseteq L(Q')$ because we cannot prove that the inclusion cannot be proper.

Negation-free and conjunction-free logic

The formulas that can be written with this logic are in the form:

$$\varphi = True \mid \ \diamond a\varphi$$

We write $L_{\neg,\wedge}(P)$ the set of formulas satisfied by P. Let also call $Lang(P)$ the set of strings accepted by the automaton isomorphic to the LTS of P where every state is final.
We can prove that:

$$L_{\neg,\wedge}(P) = L_{\neg,\wedge}(Q) \iff Lang(P) = Lang(Q)$$

Proof:
Let $\varphi = \diamond a_1, \ldots, \diamond a_n T$ and $\varphi \in L_{\neg,\wedge}(P)$. This means that:

$$\exists P_1, \ldots, P_n | P \xrightarrow{a_1} P_1 \ldots \xrightarrow{a_n} P_n$$

So, only if $a_1 \ldots a_n \in Lang(P)$.
Two processes with the same language are called **trace equivalent**, where a trace is every sequence of actions performed by the process.

3.5.2 Logic for weak bisimilarity

Logic for weak bisimilarity

The formulas that can be written with this logic are in the form:

$$\varphi = True \mid \neg\varphi \mid \varphi \wedge \varphi \mid \ \ll\gg a\varphi$$

The difference with the logic for strong bisimilarity is the $\diamond$ turned into $\ll\gg$. This is the same difference between strong and weak bisimilarity, so:

$$P \models \ll\gg a\varphi \iff \exists P' | P \stackrel{\hat{a}}{\Rightarrow} P' \wedge P' \models \varphi$$